ETSI GS NFV-IFA 002 V2.3.1 (2017-08)



Network Functions Virtualisation (NFV) Release 2; Acceleration Technologies; VNF Interfaces Specification

Disclaimer

The present document has been produced and approved by the Network Functions Virtualisation (NFV) ETSI Industry Specification Group (ISG) and represents the views of those members who participated in this ISG.
It does not necessarily represent the views of the entire ETSI membership.

Reference
RGS/NFV-IFA002ed231

Keywords
acceleration, interoperability, NFV, NFVI,
performance, portability

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - NAF 742 C
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° 7803/88

Important notice

The present document can be downloaded from:
http://www.etsi.org/standards-search

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the only prevailing document is the print of the Portable Document Format (PDF) version kept on a specific network drive within ETSI Secretariat.

Users of the present document should be aware that the document may be subject to revision or change of status.
Information on the current status of this and other ETSI documents is available at
https://portal.etsi.org/TB/ETSIDeliverableStatus.aspx

If you find errors in the present document, please send your comment to one of the following services:
https://portal.etsi.org/People/CommiteeSupportStaff.aspx

Copyright Notification

No part may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm except as authorized by written permission of ETSI.
The content of the PDF version shall not be modified without the written authorization of ETSI.
The copyright and the foregoing restriction extend to reproduction in all media.

© ETSI 2017.
All rights reserved.

DECT™, **PLUGTESTS**™, **UMTS**™ and the ETSI logo are trademarks of ETSI registered for the benefit of its Members.
3GPP™ and **LTE**™ are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners.
oneM2M logo is protected for the benefit of its Members.
GSM® and the GSM logo are trademarks registered and owned by the GSM Association.

Contents

Intellectual Property Rights 7
Foreword 7
Modal verbs terminology 7
1 Scope 8
2 References 8
2.1 Normative references 8
2.2 Informative references 8
3 Definitions and abbreviations 9
3.1 Definitions 9
3.2 Abbreviations 11
4 Overview 11
4.1 Problem Statement 11
4.1.1 VNF Acceleration goals 11
4.1.2 Network related acceleration 13
4.1.3 Storage related acceleration 13
4.1.4 Algorithmic acceleration 13
4.2 Software architecture 14
4.2.1 Overview 14
4.2.2 Acceleration model 14
4.2.2.1 General 14
4.2.2.2 VNF aspects 15
4.2.2.3 Virtualisation Layer aspects 17
4.2.2.4 Intra-VNF acceleration 18
5 Abstract Interface functional requirements 20
5.1 Overview 20
5.2 Common Acceleration Virtualisation interface requirements 21
5.3 EPD Driver requirements 21
5.4 Cryptography functional group 22
5.4.1 Overall requirements 22
5.4.2 Operations requirements 22
5.4.3 Crypto interface requirements 23
5.4.4 Crypto driver requirements 23
5.4.5 Management and monitoring requirements 23
5.5 IPSec functional group 23
5.5.1 Overview 23
5.5.2 IPSec interface requirements 24
5.5.3 Operations requirements 24
5.5.4 Management and monitoring requirements 24
5.6 TCP functional group 24
5.6.1 TCP interface requirements 24
5.6.2 TCP type requirements 25
5.7 Re-programmable computing functional group 25
5.7.1 Re-programmabe interface requirements 25
5.7.2 Operations requirements 25
5.7.3 Management and monitoring requirements 25
5.8 Dynamic Optimization of Packet Flow Routing Functional Group 26
5.8.1 DOPFR interface requirements 26
5.8.2 Management and monitoring requirements 26
5.9 NAT functional group 26
5.9.1 Overview 26
5.9.2 Overall requirements 26
5.9.3 NAT interface requirements 26
5.9.4 NAT Operations requirements 27
5.9.5 Management and monitoring requirements 27

5.10 VXLAN functional group 27
5.10.1 Overview 27
5.10.2 Overall requirements 27
5.10.3 VXLAN interface requirements 27
5.10.4 VXLAN operations requirements 27
5.10.5 Management and monitoring requirements 28
5.11 Media functional group 28
5.11.1 Overview 28
5.11.2 Media overall requirements 28
5.11.3 Media operations requirements 28
5.11.4 Media interface requirements 29
5.11.5 Management and monitoring requirements 29
5.12 QoS Functional group 29
5.12.1 Overview 29
5.12.2 QoS interface requirements 31
5.12.3 QoS Operations requirements 31
5.12.4 Management and monitoring requirements 32
5.13 PDCP Functional group 32
5.13.1 Overview 32
5.13.2 Overall requirements 32
5.13.3 Operations requirements 32
5.13.4 Management and monitoring requirements 33

6 UML description of abstract interfaces 33
6.1 General 33
6.2 Acceleration Virtualisation interface 33
6.2.1 Initialization 33
6.2.2 Configuration 34
6.2.3 Fault Management 34
6.3 EPD Driver interface 35
6.3.1 Cryptography functional group 35
6.3.1.1 Overview 35
6.3.1.2 Key generation 35
6.3.1.3 Operation execution 36
6.3.1.4 Inline packet encryption/decryption 37
6.3.2 IPSec functional group 37
6.3.3 TCP functional group 38
6.3.4 Re-programmable computing functional group 39
6.3.5 Dynamic Optimization of Packet Flow Routing Functional Group 40
6.3.6 NAT functional group 40
6.3.7 VXLAN functional group 41
6.3.8 Media functional group 41
6.3.9 QoS Functional group 43

7 VNF Interfaces Specifications 44
7.1 General 44
7.2 Conventions 44
7.3 EPD Management interface 45
7.3.1 Description 45
7.3.2 Init operation 45
7.3.2.1 Description 45
7.3.2.2 Input Parameters 45
7.3.2.3 Output Parameters 45
7.3.3 RegisterForAccEvent operation 45
7.3.3.1 Description 45
7.3.3.2 Input Parameters 46
7.3.3.3 Output Parameters 46
7.3.3.4 Operation Result 46
7.3.4 AccEventNotification operation 46
7.3.4.1 Description 46
7.3.4.2 Input Parameters 46
7.3.4.3 Output Parameters 46

7.3.4.4 Operation Result 47
7.3.5 DeRegisterForAccEvent operation 47
7.3.5.1 Description 47
7.3.5.2 Input Parameters 47
7.3.5.3 Output Parameters 47
7.3.5.4 Operation Result 47
7.3.6 ReleaseAcc operation 47
7.3.6.1 Description 47
7.3.6.2 Input Parameters 47
7.3.6.3 Output Parameters 48
7.3.6.4 Operation Result 48
7.4 EPD Configuration Interface 48
7.4.1 Description 48
7.4.2 ModifyAccConfiguration operation 48
7.4.2.1 Description 48
7.4.2.2 Input Parameters 48
7.4.2.3 Output Parameters 48
7.4.2.4 Operation Result 48
7.4.3 GetAccConfigs operation 49
7.4.3.1 Description 49
7.4.3.2 Input Parameters 49
7.4.3.3 Output Parameters 49
7.4.3.4 Operation Result 49
7.4.4 ResetAccConfigs operation 49
7.4.4.1 Description 49
7.4.4.2 Input Parameters 50
7.4.4.3 Output Parameters 50
7.4.4.4 Operation Result 50
7.5 EPD Monitoring Interface 50
7.5.1 Description 50
7.5.2 GetAccStatistics operation 50
7.5.2.1 Description 50
7.5.2.2 Input Parameters 51
7.5.2.3 Output Parameters 51
7.5.2.4 Operation Result 51
7.5.3 ResetAccStatistics operation 51
7.5.3.1 Description 51
7.5.3.2 Input Parameters 51
7.5.3.3 Output Parameters 52
7.5.3.4 Operation Result 52
7.6 EPD Data Interface 52
7.6.1 Description 52
7.6.2 Accelerate operation 52
7.6.2.1 Description 52
7.6.2.2 Input Parameters 52
7.6.2.3 Output Parameters 52
7.6.2.4 Operation Result 53
7.6.3 AccSend operation 53
7.6.3.1 Description 53
7.6.3.2 Input Parameters 53
7.6.3.3 Output Parameters 53
7.6.3.4 Operation Result 53
7.6.4 AccReceive operation 53
7.6.4.1 Description 53
7.6.4.2 Input Parameters 54
7.6.4.3 Output Parameters 54
7.6.4.4 Operation Result 54
7.6.5 RegisterForAccDataAvailableEvent operation 54
7.6.5.1 Description 54
7.6.5.2 Input parameters 55
7.6.5.3 Output parameters 55
7.6.5.4 Operation Result 55

7.6.6 AccDataAvailableEventNotification operation 55
7.6.6.1 Description 55
7.6.6.2 Input Parameters 55
7.6.6.3 Output Parameters 55
7.6.6.4 Operation Result 55
7.6.7 DeRegisterForAccDataAvailableEvent operation 56
7.6.7.1 Description 56
7.6.7.2 Input Parameters 56
7.6.7.3 Output parameters 56
7.6.7.4 Operation Result 56

8 Information elements exchanged 56
8.1 Introduction 56
8.2 Information elements and notifications related to EPD Interface Management operations 56
8.2.1 Introduction 56
8.2.2 AccCapabilities information element 56
8.2.2.1 Description 56
8.2.2.2 Attributes 57
8.2.3 SubSysCapabilities information element 57
8.2.3.1 Description 57
8.2.3.2 Attributes 57
8.2.4 AccEventType information element 57
8.2.4.1 Description 57
8.2.4.2 Attributes 57
8.2.5 AccEventMetaData information element 57
8.2.5.1 Description 57
8.2.5.2 Attributes 57
8.3 Information elements and notifications related to EPD Interface Configuration operations 58
8.3.1 Introduction 58
8.3.2 AccConfigurationType information element 58
8.3.2.1 Description 58
8.3.2.2 Attributes 58
8.3.3 AccSubSysConfigurationType information element 58
8.3.3.1 Description 58
8.3.3.2 Attributes 58
8.4 Information elements and notifications related to EPD Interface Monitoring operations 58
8.4.0 Introduction 58
8.4.1 AccStatistics information element 59
8.4.1.1 Description 59
8.4.1.2 Attributes 59
8.4.2 SubSysStatistics information element 59
8.4.2.1 Description 59
8.4.2.2 Attributes 59
8.5 Information elements and notifications related to EPD Data Interface operations 59
8.5.1 Introduction 59
8.5.2 AccDataType information element 59
8.5.2.1 Description 59
8.5.2.2 Attributes 59

Annex A (informative): Authors & contributors 61

Annex B (informative): Bibliography 62

Annex C (informative): Change History 63

History 64

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to the present document may have been declared to ETSI. The information pertaining to these essential IPRs, if any, is publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI Web server (https://ipr.etsi.org/).

Pursuant to the ETSI IPR Policy, no investigation, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

Foreword

This Group Specification (GS) has been produced by ETSI Industry Specification Group (ISG) Network Functions Virtualisation (NFV).

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

1 Scope

The present document specifies requirements for a set of abstract interfaces enabling a VNF to leverage acceleration services from the infrastructure, regardless of their implementation. The present document also provides an acceleration architectural model to support its deployment model.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

Referenced documents which are not found to be publicly available in the expected location might be found at https://docbox.etsi.org/Reference/.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long term validity.

The following referenced documents are necessary for the application of the present document.

[1] ETSI GS NFV 003: "Network Functions Virtualisation (NFV); Terminology for main concepts in NFV".

[2] ETSI TS 136 323: "LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Packet Data Convergence Protocol (PDCP) specification (3GPP TS 36.323)".

[3] ETSI TS 133 401: "Digital cellular telecommunications system (Phase 2+) (GSM); Universal Mobile Telecommunications System (UMTS); LTE; 3GPP System Architecture Evolution (SAE); Security architecture (3GPP TS 33.401)".

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long term validity.

The following referenced documents are not necessary for the application of the present document but they assist the user with regard to a particular subject area.

[i.1] ETSI GS NFV-INF 003: "Network Functions Virtualisation (NFV); Infrastructure; Compute Domain".

[i.2] ETSI GS NFV-SWA 001: "Network Functions Virtualisation (NFV); Virtual Network Functions Architecture".

[i.3] ETSI GS NFV-IFA 003: "Network Functions Virtualisation (NFV); Acceleration Technologies; vSwitch Benchmarking and Acceleration Specification".

[i.4] ETSI GS NFV-IFA 004: "Network Functions Virtualisation (NFV); Acceleration Technologies; Management aspects Specification".

[i.5] Void.

[i.6] ETSI GS NFV-INF 005: "Network Functions Virtualisation (NFV);Infrastructure; Network Domain".

[i.7] ETSI GS NFV-IFA 001: "Network Functions Virtualisation (NFV); Acceleration Technologies; Report on Acceleration Technologies & Use Cases".

[i.8] ISO/IEC 9646-7: "Information technology - Open Systems Interconnection - Conformance testing methodology and framework - Part 7: Implementation Conformance Statements".

3 Definitions and abbreviations

3.1 Definitions

For the purposes of the present document, the terms and definitions given in ETSI GS NFV 003 [1] and the following apply:

abstract interface: computer specification and modelling construct. It defines an information model and a way to communicate between two or more entities

NOTE: Computing objects and APIs can be developed for a programming language to implement it.

Application Programming Interface (API): computer programming language construct, composed of a set of functions, methods, objects, structures and constant definitions used to implement an abstract interface

NOTE: This construct is called an abstract interface language binding. There might be multiple bindings to a single abstract interface, one for each computer language (C, C++, Java™, Ruby, PHP, etc.).

Committed Burst Size (CBS): maximum volume of data the network agrees to move

Committed Information Rate (CIR): guaranteed bandwidth for traffic arriving at or departing from an interface under normal conditions

dispatching: deterministic method of distributing packets amongst packet queues to be handled by the network stack, exposed as a NIC capability

NOTE: This dispatching can be done by an operating system or a software library such as DPDK or ODP. The criteria for dispatching can be as simple as round robin or as complex as packet hashing on combination of IP address, TCP ports, taking into account tunnelling. The number of processor threads handling the queues can be lower or higher than the number of those queues.

Excess Burst Size (EBS): amount of data above and beyond the Committed Burst Size

Execution Environment (EE): set of resources and control mechanisms on which application are running

NOTE 1: Examples of Execution Environments include:

- Traditional Operating System.
- RUMP kernels.
- Java™ Virtual Machine (with or without underlying Operating System).
- Xen Minios.
- DPDK.
- Docker.

NOTE 2: An Execution Environment may be virtualised on top of a hypervisor or not.

NOTE 3: Execution Environments may share or not resources such as processor between applications running on top of them.

extensible para-virtualised device: para-virtualised device that can execute code and use resources provided by the hypervisor domain at runtime

NOTE: The benefit of the extensibility is to avoid crossing virtualisation boundary whenever it is possible. The resources enable bypassing the hypervisor in a hardware independent manner.

language binding: API definition for an abstract interface on a particular programming language

library: collection of implementations of behaviour, written in terms of a language that has a well-defined interface by which the behaviour is invoked

NOTE: This means that as long as a higher level program uses a library to make system calls, it does not need to be re-written to implement those system calls over and over again. In addition, the behaviour is provided for reuse by multiple independent programs. A program invokes the library-provided behaviour via a mechanism of the language. For example, in a simple imperative language such as C, the behaviour in a library is invoked by using C's normal function-call. What distinguishes the call as being to a library, versus being to another function in the same program, is the way that the code is organized in the system.

load balancing: dynamic application level traffic distribution function, that distributes traffic amongst VNFC instances within a VNF or amongst VNF instances

NOTE: As defined in ETSI GS NFV-SWA 001 [i.2], clause 5.1.4.

offload: delegate processing (e.g. classification, forwarding, dispatching, load balancing, cryptography, and transcoding) to a different processor or other specialized hardware entity

Peak Information Rate (PIR): burstable rate that allows bandwidth to spike beyond CIR

Real Time Application (RTA): application whose execution can be **guaranteed** to be accomplished under a specific "execution contract"

NOTE: For instance within a bounded delay, for a certain bandwidth. It assumes execution on a Real Time Execution Environment or Operating System (see Real Time Execution Environment).

Real Time Execution Environment (RTEE): Execution Environment that offer applications with provisions to meet their execution contract (see Real Time Application)

Real Time Operating System (RTOS): Real Time Execution Environment in the form of an Operating System

NOTE: An RTOS has an advanced algorithm for scheduling. Scheduler flexibility enables a wider, computer-system orchestration of process priorities, but a real-time OS is more frequently dedicated to a narrow set of applications. Key factors in a real-time OS are minimal interrupt latency and minimal thread switching latency; a real-time OS is valued more for how quickly or how predictably it can respond than for the amount of work it can perform in a given period of time.

software framework: abstraction in which software providing generic functionality can be selectively changed by additional user-written code, thus providing application-specific software

NOTE 1: A software framework is a universal, reusable software environment that provides particular functionality as part of a larger software platform to facilitate development of software applications, products and solutions. Software frameworks may include support programs, compilers, code libraries, tool sets, and application programming interfaces (APIs) that bring together all the different components to enable development of a project or solution.

NOTE 2: Frameworks contain key distinguishing features that separate them from normal libraries:

- Inversion of control: In a framework, unlike in libraries or normal user applications, the overall program's flow of control is not dictated by the caller, but by the framework.
- Default behaviour: A framework has a default behaviour. This default behaviour is some useful behaviour and not a series of no-ops.
- Extensibility: A framework can be extended by the user usually by selective overriding or specialized by user code to provide specific functionality.
- Non-modifiable framework code: The framework code, in general, is not supposed to be modified, while accepting user-implemented extensions. In other words, users can extend the framework, but should not modify its code.

3.2 Abbreviations

For the purposes of the present document, the abbreviations given in ETSI GS NFV 003 [1] and the following apply:

API	Application Programming Interface
Encrypt/Decrypt	Encryption and Decryption
Encap/Decap	Encapsulation and Decapsulation
EPD	Extensible Para-virtualised Device
NAT	Network Address Translation
NIC	Network Interface Card
SA	Security Association
SPD	Security Policy Database
UML	Unified Modelling Language
VA	Virtual Accelerator
VNI	VXLAN Network Identifier
VTEP	VXLAN Tunnel End Point
VXLAN	Virtual eXtensible Local Area Network

4 Overview

4.1 Problem Statement

4.1.1 VNF Acceleration goals

The goals of the present document are:

- to identify common design patterns that enable an executable VNFC to leverage, at runtime, accelerators to meet their performance objectives;
- to describe how a VNF Provider might leverage those accelerators in an implementation independent way; and
- to define methods in which all aspects of the VNF (VNFC, VNFD, etc.) could be made independent from accelerator implementations.

VNF providers have to mitigate two goals:

- VNFs might have constraints to perform their function within certain power consumption boundaries, CPU core count, PCI express slot usage and with good price/performance ratio; and
- VNFs should accommodate most if not all deployment possibilities.

Use of accelerators can help meet the constraints but can have an influence on deployment flexibility. VNF acceleration implementations will range from inflexible software that is tightly-coupled to specific software and hardware in the VNF and NFVI (see pass-through model as shown in figure 4.1.1-1), to highly flexible loosely-coupled software that uses common abstractions and interfaces (see abstracted model as shown in figure 4.1.1-2). Further it is understood that virtualisation and acceleration technologies will evolve. Pass-through deployments are expected to exist, and the present document does not intend to preclude any specific acceleration architectures from NFV deployments. However, the focus of the present document is to define and promote abstracted models.

It is desirable that the use of accelerators be implementation independent. There is a slight difference between "implementation independent executable VNFC" and "implementation independent VNF":

- An implementation independent executable VNFC is a software that can leverage a known set of accelerator implementations both in hardware and in software. The part of the VNFD that applies to this VNFC contains information elements that allow the NFV management and orchestration to find a compute node with the requested characteristics or hardware. Should a new hardware become available on the market, a VNF Provider willing to make use of it to accelerate a VNFC has to update it's the VNFC image and the VNFD, the operator has to on-board the new VNF package and redeploy it to make use of the new hardware. This was defined as the pass-through model in ETSI GS NFV-INF 003 [i.1], clause 7.2.2.

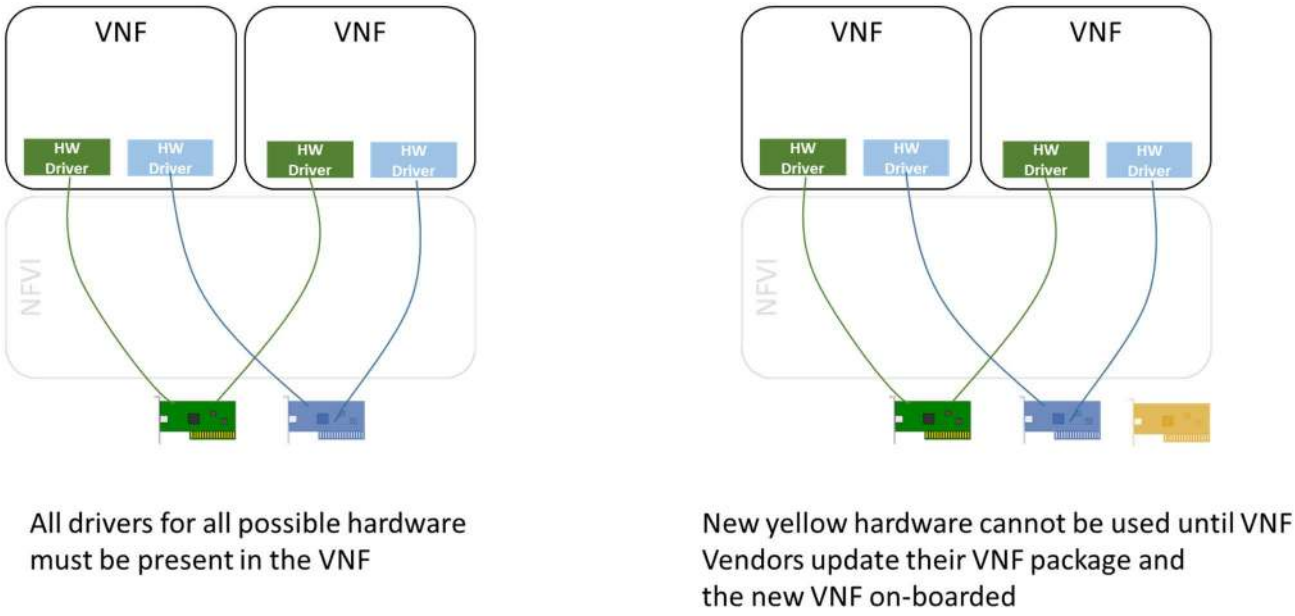


Figure 4.1.1-1: Pass-through model

- An implementation independent VNF is a VNF that makes no assumption whatsoever on the underlying NFVI. Its VNFD does not contain any accelerator specific information elements. Should a new hardware become available on the market, the operator will update its NFVI to allow the VNF to make use of the new hardware. An implementation independent VNF is thus based on implementation independent VNF software that makes use of a functional abstraction of an accelerator supported by an adaptation layer in the NFVI. This model is close to the abstracted model defined in ETSI GS NFV-INF 003 [i.1], clause 7.2.2.

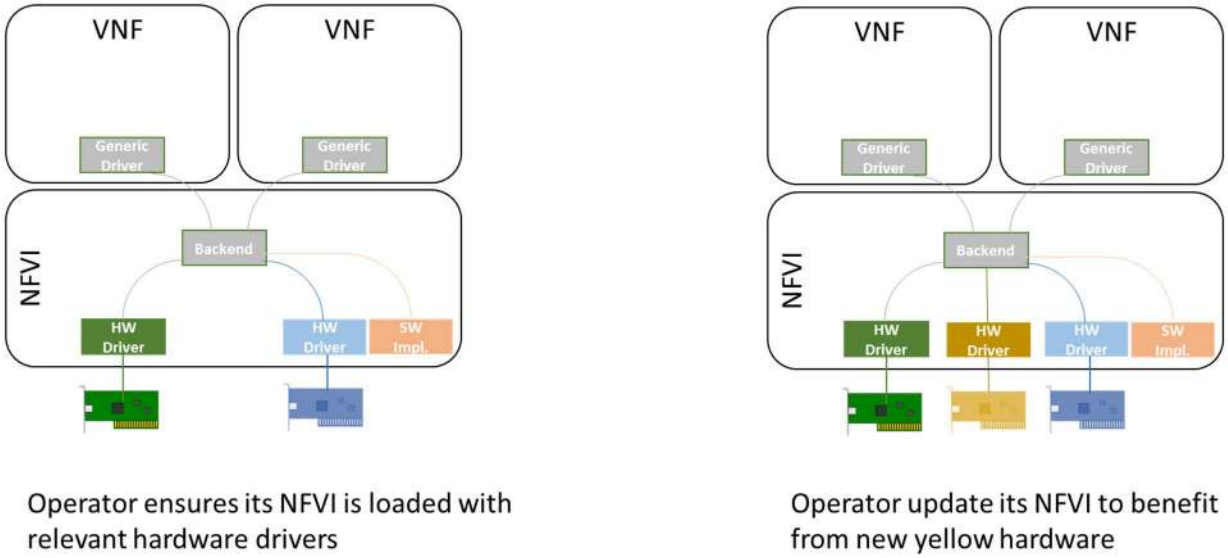


Figure 4.1.1-2: Abstracted model

NOTE 1: An implementation independent executable VNFC allows for VNF deployment in both hypervisor based and non-hypervisor based environments. The latter configuration is outside the scope of the present document.

Live migration of such hardware independent accelerated VNF may be possible if any associated acceleration state information required can also be migrated.

NOTE 2: Live migration from a compute node with accelerator to a compute node without accelerator or with a different accelerator is allowed (in particular to cope with emergency response situations).

A further refinement of the aforementioned goals is to make sure that VNFs can leverage those accelerators regardless of:

- the diversity of VNF software execution environments:
  - Operating Systems (Windows®, Linux®, etc.);
  - Java™ Virtual Machine;
  - RUMP Kernels;
  - DPDK, ODP;
- the diversity of software frameworks or libraries for acceleration, each having provisions to leverage software or hardware acceleration technologies; and
- the diversity of virtualisation environments such as KVM, Xen, VMWare ESX or Microsoft® Hyper-V.

NOTE 3: The present document focuses on acceleration of the VNF execution, it does not cover how a VNF influences packet forwarding in the NFV,which is covered by ETSI GS NFV-IFA 003 [i.3] or by ETSI GS NFV-IFA 004 [i.4]. Requirements for the packet dispatching control interface are however specified by the present document.

NOTE 4: "Microsoft®, Windows®, Linux® and Java™ are examples of a suitable products available commercially. This information is given for the convenience of users of the present document and does not constitute an endorsement by ETSI of these products".

The accelerators considered span several different execution aspects:

- network stack acceleration: Packet dispatching, manycasting, partial networking stack offloads (L4, IPSec, etc.);
- network payload acceleration: compression, transcoding, pattern matching;
- cryptography (encryption, hashing, random number generation, etc.);
- storage;
- digital Signal Processing; and
- algorithmic Acceleration.

4.1.2 Network related acceleration

The network related accelerations have to take into account the network overlays involved. While many NICs offer TCP accelerations (checksum calculation and verification, TCP Segmentation Offload, etc.) some cannot operate when the traffic is in overlay networks (either layer 2 such as VxLAN or layer 3). Furthermore, when the traffic itself is tunnelled, for instance GTP for a GGSN node, TCP accelerations on such traffic is to be considered as partial networking stack offload.

4.1.3 Storage related acceleration

There is local storage on the compute node and remote storage on other NFVI nodes. The acceleration considers how both types of storage are accessed from the VNF.

4.1.4 Algorithmic acceleration

Algorithmic acceleration can be used by VNFs and NFVI within an NFV environment in order to achieve a better performance and more deterministic behaviour when executing algorithmically complicated functions.

4.2 Software architecture

4.2.1 Overview

The industry already successfully implemented an abstraction layer for networking and storage in the form of virtio-net and virtio-block that works across virtualisation environments and across software libraries:

- A VNF leveraging virtio-net ports and virtio-block disk controllers can be live migrated between servers running different virtualisation environments.
- A VNF does not have to be updated as new networking or disk hardware is available on the market.
- Microsoft® Windows®, Apple® OS/X, Linux®, DPDK, ODP, Netmap can make use of virtio-net for accessing network packets.

NOTE: "Apple® is an example of a suitable product available commercially. This information is given for the convenience of users of the present document and does not constitute an endorsement by ETSI of this product".

The present document extends this model to specify an acceleration architectural model for NFV.

The NFV architectural framework can include a Virtual Accelerator in addition to Virtual Compute, Virtual Storage and Virtual Network that can abstract the underlying proprietary vendor-specific hardware offload accelerators and provide a virtual instance that the VNFs can use. The Virtual Accelerator shall provide a standardized vendor agnostic accelerator interface that VNFs can use to access the underlying accelerator.

4.2.2 Acceleration model

4.2.2.1 General

The acceleration model, as depicted in figure 4.2.2.1-1, is based on Extensible Paravirtualised Devices (EPD) implements one or more instances of Virtual Accelerators and is used by applications through an EPD driver running in the VNF's Execution Environment (EE) that exposes the abstract interface of a VA.

An EPDrelies on collaboration with an EPD backend executing in the Execution Environment of the NFVI (i.e. hypervisor domain) to benefit from the acceleration.

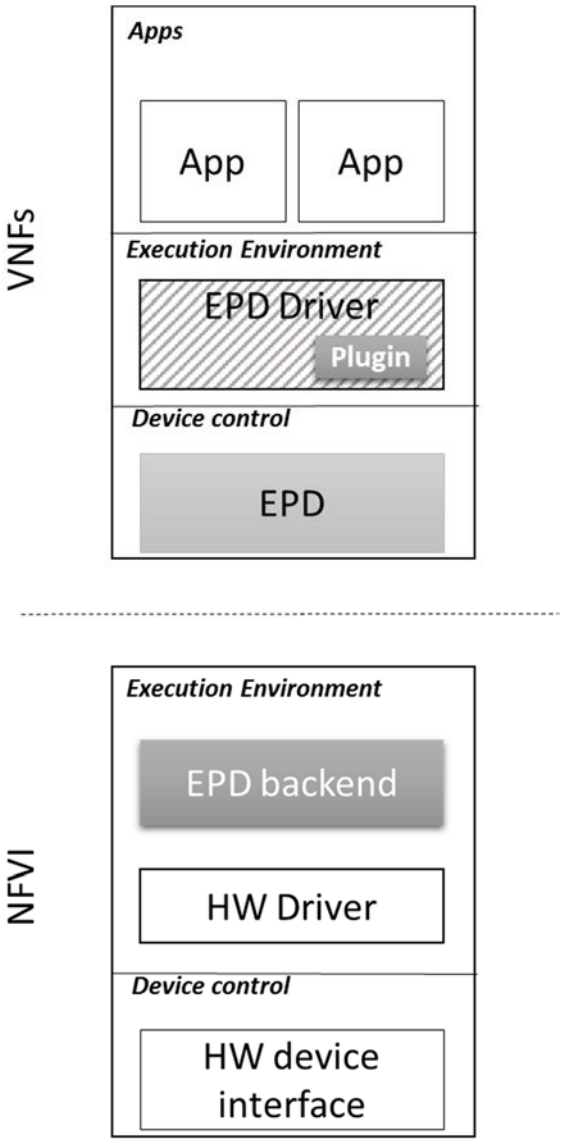


Figure 4.2.2.1-1: Acceleration model

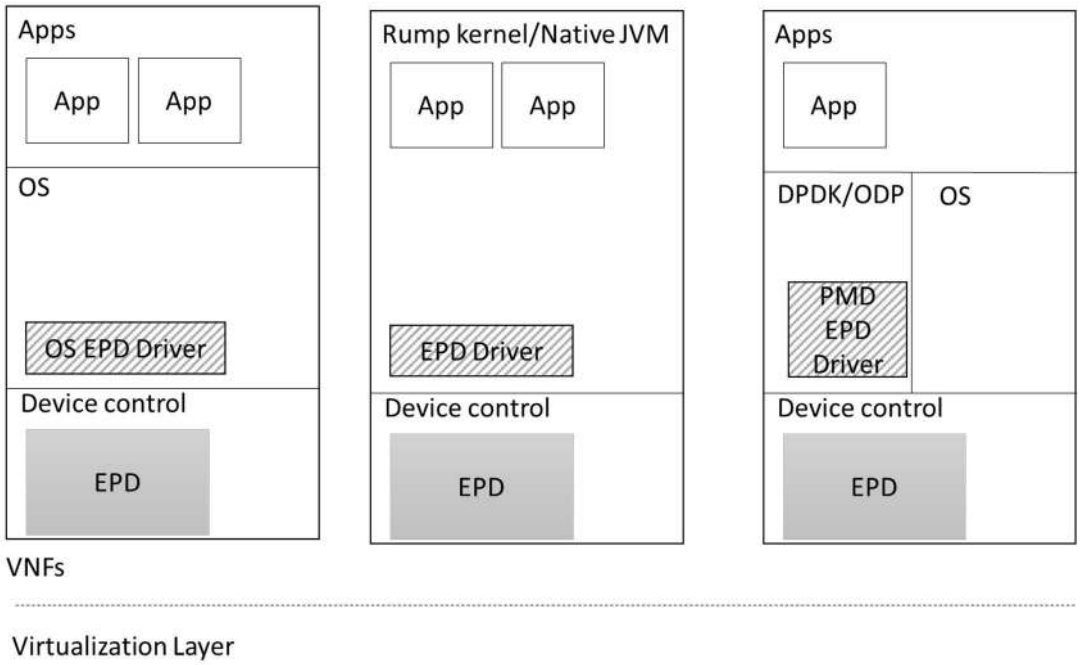
The acceleration model allows a single HW device to expose multiple VAand multiple instances of VA per VNF.

The backend may provide EPD drivers with plugins to allow intra-VNF acceleration that avoid virtualisation barrier crossing. A VNF may - but need not - use this plugin depending on security or administrative policies.

4.2.2.2 VNF aspects

For each specified Virtual Accelerator, the VNFCs to be accelerated shall include an EPD driver that provides the an abstract interface to the EPD.

The VNFC's Execution Environment implements EPD drivers for each EPD it supports and makes it available to applications through an acceleration API or framework (see figure 4.2.2.2-1).



NOTE: The "Device Control" is an interface that is used to communicate/control devices, real or virtual hardware (PCI configuration is one example of such device control interface). The "Device Control" of a VM is created and maintained the hypervisor. The "Device Control" of a server is made available to software by the processor. What happens when data is read or written to this space depends on many aspects, it can be trapped by the hypervisor itself or QEMU or other software.

Figure 4.2.2.2-1: Acceleration model - VNF part

4.2.2.3 Virtualisation Layer aspects

A Virtual Accelerator backend is associated with one or more EPSs from one or more types. It implements an adaptation layer to the hardware or to hypervisor domain resources. For instance, in the case of virtual NIC, the EPD backend does not pass the packets to the hardware but rather to a software stack (either host networking stack or other optimized data path).

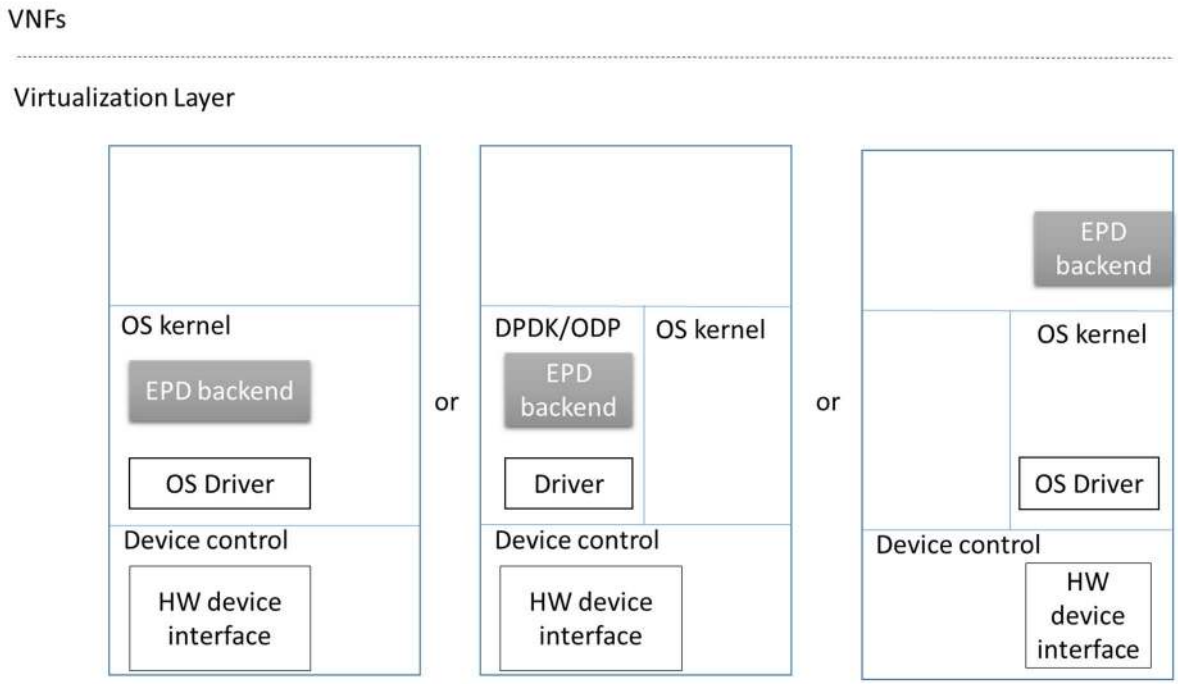


Figure 4.2.2.3-1: Acceleration model - hypervisor part

As illustrated in figure 4.2.2.3-1, the EPD backend may:

- implement acceleration directly in software;
- pass acceleration software plugin to the virtual accelerator driver; or
- drive acceleration hardware regardless of its housing/location.

The acceleration hardware can be collocated on the same compute node or located on a remote node as a network attached accelerator (for instance CloudRAN signal processor pool).

NOTE: The nature of the remote node, a "Physical Network Function" or any other entity, is for further study.

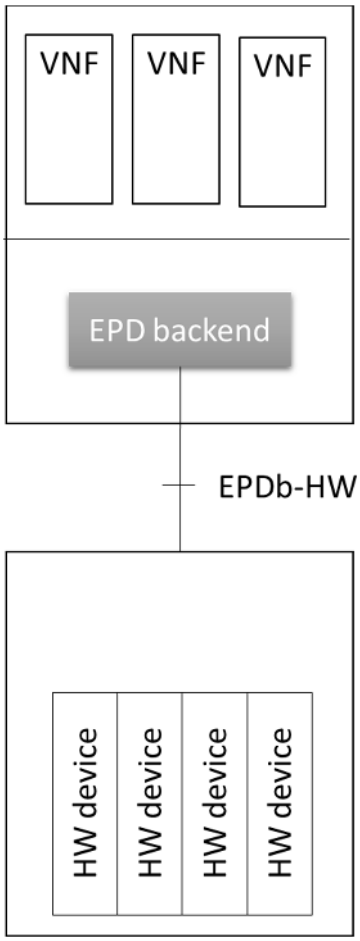


Figure 4.2.2.3-2: Acceleration housing

The EPDb-HW interface between the EPD backend and the actual hardware, shown in figure 4.2.2.3-2, is outside the scope of ETSI NFV standardization.

4.2.2.4 Intra-VNF acceleration

There are two acceleration types that can be performed within a VNF.

The first type is intra-VNFC acceleration and accelerates a VNFCI. When the acceleration is provided by resources directly accessible by the processor (collocated GPU cores, DSP, FPGA), the acceleration model shall allow direct acceleration leverage without crossing the virtualisation boundary.

In compliance to the problem statement, the EPD driver shall not include any processor specific acceleration, but rather the Virtualisation Layer shall provide the EPD at runtime with relevant software plugin so that it can leverage the acceleration.

The VNF can define security or administrative policy regarding the usage of such software plugin.

NOTE 1: The model is quite similar to Linux® VDSO model.

The second type is inter-VNFC acceleration. It can modify how the traffic is routed or forwarded between VNFCIs and may include performing additional acceleration on packet forwarding on the infrastructure network resources. This is described in ETSI GS NFV-INF 005 [i.6] as Dynamic Optimization of Packet Flow Routing (DOPFR).

The acceleration is performed by the infrastructure network resources under the control of an infrastructure network control function. For example, the infrastructure Network Controller provides logical switch constructs, one per VNFCI, for its exclusive use to offload packet processing. The logical switch may share underlying infrastructure network resources with other logical switches. The infrastructure network controller is responsible for partitioning the infrastructure network resources into logical switches. The operations performed on the logical switch by the VNFCI are translated into requests on the underlying infrastructure network resources by the infrastructure network control function.

NOTE 2: The logical switch Lifecycle Management is described in ETSI GS NFV-IFA 004 [i.4].

NOTE 3: The infrastructure network control function may but need not be in the Network Controller.

Figure 4.2.2.4-1 shows an example VNF with inter-VNFC acceleration where the red path shows the non-accelerated packet flow and the blue path shows the accelerated packet flow.

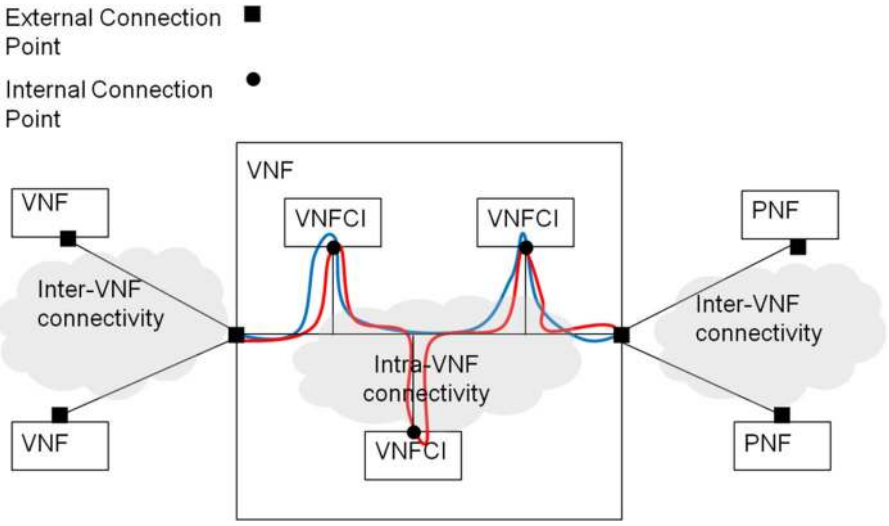


Figure 4.2.2.4-1: Inter-VNFC acceleration

NOTE 4: The acceleration boundaries consist of the VNF external connection points.

Figure 4.2.2.4-2 shows an example logical switch representation for figure 4.2.2.4-1.

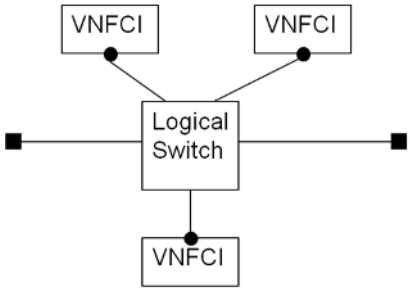


Figure 4.2.2.4-2: Logical switch representation for figure 4.2.2.4-1

Figure 4.2.2.4-3 shows another example VNF with inter-VNFC acceleration where the red path shows the non-accelerated packet flow and the blue path shows the accelerated packet flow. Note that the need to modify the intra-VNF connectivity in order to provide acceleration in the infrastructure network requires the external endpoints to have virtual links to the internal connection points of a VNFCI.

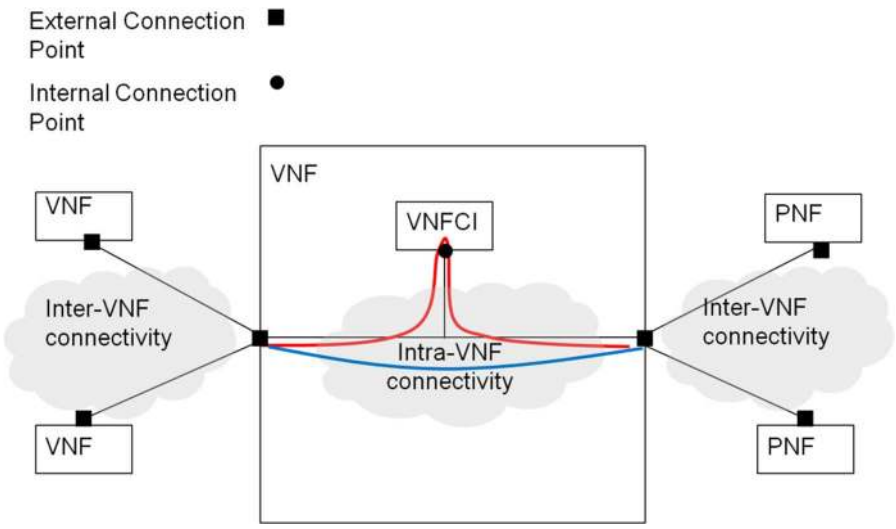


Figure 4.2.2.4-3: Inter-VNFC acceleration with single VNFCI

Figure 4.2.2.4-4 shows an example logical switch representation for figure 4.2.2.4-3.

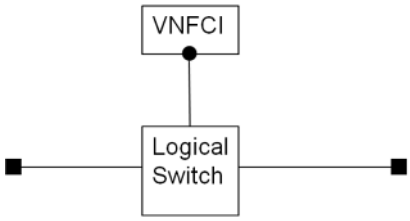


Figure 4.2.2.4-4: Logical switch representation for figure 4.2.2.4-3

5 Abstract Interface functional requirements

5.1 Overview

Virtual Accelerator is made visible and used by the EPD Driver over the Vn-Nf reference point. In order to achieve this visibility, control and usage technologies such as virtio can be used.

ETSI GS NFV-IFA 001 [i.7] lists a number of use cases that define many different accelerators covering very diverse functional domains such as cryptography and video transcoding.

An EPD Driver is integrated in VNF specific Execution Environments, so their structure and north bound interfaces or APIs are out of scope of the present document. The EPD Driver shall still implement a defined behaviour and state machine when leveraging any accelerator.

Clause 5.2 covers generic requirements for all accelerators and clause 5.3 covers requirements applicable to the EPD drivers. Subsequent clauses define functional requirements for each selected accelerator from ETSI GS NFV-IFA 001 [i.7] use cases. All interface requirements specified from clause 5.4 onwards apply to both the northbound interfaces of an EPD driver (i.e. between the EPD driver and applications) and the southbound interface of an EPD (i.e. between the EPD and the EPD backend).

5.2 Common Acceleration Virtualisation interface requirements

Table 5.2-1

Numbering	Functional requirements description
VnNf.Accel.001	Acceleration Virtualisation interface shall support exchanging data (packets, frames, etc.) in both directions.
VnNf.Accel.002	Acceleration Virtualisation interface shall support multiple channels of communication. Channels are independent communication instances across the Acceleration Virtualisation interface. Examples of channels are virtual functions, DMA queues, ports and sockets.
VnNf.Accel.003	Acceleration Virtualisation interface shall support control usage of channels (e.g. priority, number of packets per second).
VnNf.Accel.004	Acceleration Virtualisation interface shall support scaling of VNF/VNFC.
VnNf.Accel.005	Acceleration Virtualisation interface shall support notification of multiple events for the same EPD.
VnNf.Accel.006	Acceleration Virtualisation interface shall support reading capabilities and capacities of accelerator.
VnNf.Accel.007	Acceleration Virtualisation interface shall support sending and reading configuration of accelerator.
VnNf.Accel.008	Acceleration Virtualisation interface shall support obtaining EPD plugin.
VnNf.Accel.009	Acceleration Virtualisation interface shall support state transfer (e.g. for migration) in both direction.
VnNf.Accel.010	Acceleration Virtualisation interface shall support reading statistical information of accelerator.
VnNf.Accel.011	Acceleration Virtualisation interface shall support notifying drivers with new version of EPD plugin.
VnNf.Accel.012	Acceleration Virtualisation interface shall support notifying drivers with live migration events.
VnNf.Accel.013	Acceleration Virtualisation interface shall support device lifecycle management.
VnNf.Accel.014	Acceleration Virtualisation interface shall support unidirectional and bidirectional channels.
VnNf.Accel.015	Acceleration Virtualisation interface shall support synchronous and asynchronous channels.
VnNf.Accel.016	Acceleration Virtualisation interface shall support QoS.
VnNf.Accel.017	Acceleration Virtualisation interface shall support one or more context.

5.3 EPD Driver requirements

Table 5.3-1

Numbering	Functional requirements description
VnNf.Drv.001	EPD Driver shall check version compatibility with EPD backend.
VnNf.Drv.002	EPD Driver shall check existence through the Acceleration Virtualisation Interface of plugins to be executed in the VNF context.
VnNf.Drv.003	EPD Driver shall consume (that is remove from notification queue) and optionally respond or react to Acceleration Virtualisation Interface notifications.
VnNf.Drv.004	EPD Driver shall adapt to scaling requests.
NOTE: The originator of the scaling requests to EPD Driver is not identified at the time of writing and is for further study.	

5.4 Cryptography functional group

5.4.1 Overall requirements

Table 5.4.1-1

Numbering	Functional requirements description
Crypto.001	Crypto interface shall support acceleration of defined minimum set of IPSec related cryptography operations.
Crypto.002	Crypto interface shall support acceleration of defined minimum set of SSL related cryptography operations.
Crypto.003	Crypto interface shall support acceleration of defined minimum set of TLS 1.0 and above related cryptography operations.
Crypto.004	Crypto interface shall support acceleration of defined minimum set of HTTP2.0 related cryptography operations.
Crypto.005	Crypto interface shall support acceleration of defined minimum set of SRTP related cryptography operations.
Crypto.006	Crypto interface shall support acceleration of defined minimum set of SRTCP related cryptography operations.
Crypto.007	Crypto interface shall support acceleration of cryptographic primitive operations, e.g. key generation of x bits length, obtain random number in range x…y with distribution z (Gaussian, flat, etc.), obtain prime number with bit length x.

5.4.2 Operations requirements

Table 5.4.2-1

Numbering	Functional requirements description
CryptoOps.001	Crypto interface shall support symmetric cryptography operations for a defined, minimal set of symmetric cryptographic functions, e.g. Feistel based functions.
CryptoOps.002	Crypto interface shall support asymmetric cryptography operations for a defined, minimal set of asymmetric cryptographic functions.
CryptoOps.003	Crypto interface shall support random number generation operations.
CryptoOps.003.1	Data/Metadata about the random number generation should be available, e.g. its verification/trust, provenance and properties of seed values, performance characteristics (i.e. rate of random number generation).
CryptoOps.004	Crypto interface shall support obtaining the list of available crypto algorithms.
CryptoOps.005	Crypto interface shall support asynchronous operations.
CryptoOps.006	Crypto interface shall support synchronous operations.
CryptoOps.007	Crypto interface shall support cryptography operations whose context is remotely stored in the accelerator (stateful acceleration, or more precisely functional offload).
CryptoOps.008	Crypto interface shall support cryptography operations whose context is locally stored in the VNF (stateless acceleration).
CryptoOps.009	Crypto interface shall support obtaining the cryptographic key.

5.4.3 Crypto interface requirements

Table 5.4.3-1

Numbering	Functional requirements description
VnNf.crypto.001	Crypto interface shall support cryptography session lifecycle management (initialization, configuration, reconfiguration, termination, etc.).
VnNf.crypto.002	Crypto interface shall support notification of cryptographic operations completion.
VnNf.crypto.003	Crypto interface shall support obtaining the list of available hashing encryption and key generation algorithms and random number generators.
VnNf.crypto.003.1	Crypto interface shall support obtaining list of available operations provided by the accelerator: these shall include at least the hashing, encryption and key generation and random number generators.
VnNf.crypto.004	Crypto interface shall support querying the progress of ciphering, hashing, random number generation, key generation, etc. operations.
VnNf.crypto.005	Crypto interface shall support handling multi-part buffers (for instance for IP fragments or SSL blocks).
VnNf.crypto.006	Crypto interface shall support notification of cryptographic events such as IPSec re-keying requests.
VnNf.crypto.007	Crypto interface shall support notification of cryptographic alarms such as IPSec stop because of SA (security association) hard volume overflow without re-keying.
VnNf.crypto.008	Crypto interface shall support obtaining the list of available cipher algorithms and modes.

5.4.4 Crypto driver requirements

Table 5.4.4-1

Numbering	Functional requirements description
CryptDrv.001	Crypto driver shall support execution of virtualisation supplied plugin for intra-VNF acceleration (based on instruction or CPU collocated resources such as GPU and FPGA).

5.4.5 Management and monitoring requirements

Table 5.4.5-1

Numbering	Functional requirements description
CryptMgmt.001	Crypto interface shall support retrieving and resetting cryptography statistics.
CryptMgmt.002	Crypto interface shall support notification of threshold crossing.
CryptMgmt.003	Crypto interface shall support initializing defined minimum set of security context and parameters (e.g. SA, public/private keys, certificates).
CryptMgmt.004	Crypto interface shall support retrieving possible thresholds.
CryptMgmt.005	Crypto interface shall support retrieving a set of supported parameters.
CryptMgmt.006	Crypto interface shall support the irrecoverable wiping of all stored and/or provisioned information in the cryptographic accelerator (i.e. a complete, proper, unrecoverable wipe) - either in a given context or the unit as a whole.
CryptMgmt.007	Crypto accelerator interface should support trusted computing capabilities, e.g. to verify firmware of the hardware accelerator; and other trust/signing measurement capabilities, e.g. Trusted Platform Module.

5.5 IPSec functional group

5.5.1 Overview

IPSec uses cryptographic security services to protect communications over IP network. IPSec Interface aims to offload the packets security processing to the accelerator who knows how to do that from the Security Association (SA) between the sender and receiver. The SA is established by the negotiation of cryptography keys which can be accelerated via related cryptography acceleration.

5.5.2 IPSec interface requirements

Table 5.5.2-1

Numbering	Functional requirements description
VnNF.IPSec.001	IPSec interface shall support reception of SA information.
VnNF.IPSec.002	IPSec interface shall support obtaining traffic forwarding policy, such as flow-based forwarding, subnet-based forwarding.
VnNF.IPSec.003	IPSec interface shall support notification of unmatched traffic against the forwarding policy.

5.5.3 Operations requirements

Table 5.5.3-1

Numbering	Functional requirements description
IPSec.Ops.001	IPSec interface shall support flow classification based on traffic forwarding policy.
IPSec.Ops.002	IPSec interface shall support checking ingress traffic against the SPD.
IPSec.Ops.003	IPSec interface shall support checking ingress traffic whether maps to an existing SA.
IPSec.Ops.004	IPSec interface shall support notification of creating a new SA if no SA is mapped for ingress traffic.
IPSec.Ops.005	IPSec interface shall support Encrypt/Decrypt operations with mapped SA.
IPSec.Ops.006	IPSec interface shall support authentication operations with mapped SA.
IPSec.Ops.007	IPSec interface shall support Encap/Decap operations with given SA.

5.5.4 Management and monitoring requirements

Table 5.5.4-1

Numbering	Functional requirements description
IPSec.Mgmt.001	IPSec interface shall support reporting active flows statistics.
IPSec.Mgmt.002	IPSec interface shall support reporting inactive flows statistics.

5.6 TCP functional group

5.6.1 TCP interface requirements

Table 5.6.1-1

Numbering	Functional requirements description
TCP.001	TCP interface shall support enabling and disabling Partial TCP offload.
TCP.002	TCP interface shall support passing TCP connection states to ToE after TCP connection is setup in case Partial TCP offload is enabled.
TCP.003	TCP interface shall support releasing TCP connection states to host CPU after TCP connection is closed in case Partial TCP offload is enabled.
TCP.004	TCP interface shall support releasing TCP connection states to host CPU in case Partial TCP offload is disabled.
TCP.005	TCP interface shall support notifying ToE which TCP connection will be offloaded.
TCP.006	TCP interface shall support notifying ToE which TCP connection will be retrieved.

5.6.2 TCP type requirements

Table 5.6.2-1

Numbering	Functional requirements description
VnNF.TCP.001	TCP interface shall support TCP checksum offload.
VnNF.TCP.002	TCP interface shall support large segment offload.
VnNF.TCP.003	TCP interface shall support large receive offload.
VnNF.TCP.004	TCP interface shall support TCP acknowledgement offload.

5.7 Re-programmable computing functional group

5.7.1 Re-programmabe interface requirements

Table 5.7.1-1

Numbering	Functional requirements description
VnNf.ReProg.001	Re-programmable interface shall support obtaining the capabilities of the platform (Memory, DMA, etc.).
VnNf.ReProg.002	Re-programmable interface shall support passing a compilation image.
VnNf.ReProg.003	Re-programmable interface shall support multiple computing functions.
VnNf.ReProg.004	Re-programmable interface shall support obtaining the capabilities of a particular function.
VnNf.ReProg.005	Re-programmable interface shall support memory creation in target, memory freeing, and memory copying.
VnNf.ReProg.006	Re-programmable interface should support coherent memory.
VnNf.ReProg.007	Re-programmable interface shall support transferring data to particular memory location.
VnNf.ReProg.008	Re-programmable interface shall support transferring data in a streaming mode fashion to a particular function.

5.7.2 Operations requirements

Table 5.7.2-1

Numbering	Functional requirements description
ReProgOps.001	Re-programmable interface shall support obtaining the state of a particular function.
ReProgOps.002	Re-programmable interface shall support starting/stopping a particular function.
ReProgOps.003	Re-programmable interface shall support retrieving completion events from a particular function.
ReProgOps.004	Re-programmable interface shall support retrieving various notification types from a particular function.
ReProgOps.005	Re-programmable interface shall support changing the state of available functions.

5.7.3 Management and monitoring requirements

Table 5.7.3-1

Numbering	Functional requirements description
ReProgMgmt.001	Re-Programmable interface shall support initializing and configuring platforms.
ReProgMgmt.002	Re-Programmable interface shall support retrieving and resetting platform statistics.
ReProgMgmt.003	Re-Programmable interface shall support initializing and configuring particular function.
ReProgMgmt.004	Re-Programmable interface shall support retrieving and resetting particular function statistics.
ReProgMgmt.005	Re-Programmable interface shall support debugging a particular function (step by step debug).

5.8 Dynamic Optimization of Packet Flow Routing Functional Group

5.8.1 DOPFR interface requirements

Table 5.8.1-1

Numbering	Functional requirements description
VnNF.DOPFR.001	VNF DOPFR interface shall support discovering if the infrastructure network has the optimization capability.
VnNF.DOPFR.002	VNF DOPFR interface shall support instructing the infrastructure network which packet flow will be optimized.
VnNF.DOPFR.003	VNF DOPFR interface shall support instructing the infrastructure network how the packet flow is forwarded, e.g. via forwarding graph.
VnNF.DOPFR.004	VNF DOPFR interface shall support instructing the infrastructure network which packet flow will be routed back to VNF.

5.8.2 Management and monitoring requirements

Table 5.8.2-1

Numbering	Functional requirements description
DOPFR.Mgmt.001	VNF DOPFR interface shall support report of the optimized flow statistics to VNF.

5.9 NAT functional group

5.9.1 Overview

The following requirements apply to both the use of NAT within the NFVI as part of the vSwitch, and the customer specific traffic separated from the provider traffic.

5.9.2 Overall requirements

Table 5.9.2-1

Numbering	Functional requirements description
NAT.001	NAT function shall support acceleration of network address translation of source IP address, destination IP address, L4 source port number, L4 destination port number.

5.9.3 NAT interface requirements

Table 5.9.3-1

Numbering	Functional requirements description
NATif.001	NAT interface shall support obtaining of NAT policy, such as static mapping and dynamic mapping.
NATif.002	NAT interface shall support reception of NAT method, such as full-cone NAT, restricted-cone NAT, symmetric NAT.
NATif.003	NAT interface shall support reception of translation information for forward and reverse translation flows.
NATif.004	NAT interface shall support obtaining the state of NAT session.

5.9.4 NAT Operations requirements

Table 5.9.4-1

Numbering	Functional requirements description
NATops.001	NAT interface shall support notification of the change of the state of NAT session.

5.9.5 Management and monitoring requirements

Table 5.9.5-1

Numbering	Functional requirements description
NATmgmt.001	NAT interface shall support retrieving and resetting NAT statistics.
NATmgmt.002	NAT interface shall support initializing defined minimum set of NAT session.

5.10 VXLAN functional group

5.10.1 Overview

The following requirements apply to both the use of VXLAN within the NFVI as part of the vSwitch, and the customer specific traffic separated from the provider traffic.

5.10.2 Overall requirements

Table 5.10.2-1

Numbering	Functional requirements description
VXLAN.001	VXLAN interface shall support acceleration of packet encapsulation and decapsulation with VXLAN header by MAC-in-UDP.

5.10.3 VXLAN interface requirements

Table 5.10.3-1

Numbering	Functional requirements description
VXLANif.001	VXLAN interface shall support reception of Encap/Decap information between source VTEP and destination VTEP, such as VLAN,VNI,IP address of VTEP.
VXLANif.002	VXLAN interface shall support reception of mapping information between VLAN ID and VNI.
VXLANif.003	VXLAN interface shall support obtaining traffic forwarding policy, such as IP-based forwarding, MAC-based forwarding.

5.10.4 VXLAN operations requirements

Table 5.10.4-1

Numbering	Functional requirements description
VxLANops.001	VXLAN interface shall support decapsulation operations of VLAN and encapsulation operations with VXLAN header by MAC-in-UDP.
VXLANops.002	VXLAN interface shall support decapsulation operations with VXLAN header by MAC-in-UDP and encapsulation operations with VLAN.
VXLANops.003	VXLAN interface shall maintain the mappings of VLAN IDs to VNIs.

5.10.5 Management and monitoring requirements

Table 5.10.5-1

Numbering	Functional requirements description
VXLANmgmt.001	VXLAN interface shall support retrieving and resetting statistics of VXLAN information.

5.11 Media functional group

5.11.1 Overview

Multimedia communication services are widely used, which mainly are video services and audio services. Users use a variety of terminals such as TV sets, computers, smart phones to access the communication services. For the interoperability between terminals and requirements of services, VNFs should provide various media-processing capabilities, such as media transcoding, mixing, etc. This is because the amount of media is much greater, the requirements for real-time processing and latency are more stringent, and media accelerators are needed.

In addition to the transcoding operation which is described in ETSI GS NFV-IFA 001 [i.7], clause 5.1.5, there are also several basic media acceleration operations:

- Video and audio mixing in the conference service.
- Video encoding and decoding in multimedia service.

5.11.2 Media overall requirements

Table 5.11.2-1

Numbering	Functional requirements description
Media.001	Media interface shall support acceleration of defined minimum set of video related operations.
Media.002	Media interface shall support acceleration of defined minimum set of audio related operations.

5.11.3 Media operations requirements

Table 5.11.3-1

Numbering	Functional requirements description
MediaOps.001	Media interface shall support video transcoding (e.g. codec, video resolutions, resizing, horizontal stretching) operations.
MediaOps.002	Media interface shall support audio transcoding (e.g. codec) operations.
MediaOps.003	Media interface shall support video decoding operations.
MediaOps.004	Media interface shall support video encoding operations.
MediaOps.005	Media interface shall support video mixing operations (e.g. composite multiple video sources with effects into a single canvas).
MediaOps.006	Media interface shall support audio mixing operations.

5.11.4 Media interface requirements

Table 5.11.4-1

Numbering	Functional requirements description
VnNf.Media.001	Media interface shall support synchronous operations.
VnNf.Media.002	Media interface shall support asynchronous operations.
VnNf.Media.003	Media interface shall support notification of media (audio and video) operations completion.
VnNf.Media.004	Media interface shall support querying the progress of media (audio and video) operations.
VnNf.Media.005	Media interface shall support acceleration of concurrent media (audio and video) operations.
VnNf.Media.006	Media interface shall support obtaining the list of available media formats.

5.11.5 Management and monitoring requirements

Table 5.11.5-1

Numbering	Functional requirements description
MediaMgmt.001	Media interface shall support obtaining the utilization efficiency of media acceleration resources, e.g. usage of the media acceleration resources for audio transcoding, etc.
MediaMgmt.002	Media interface shall support obtaining the statistics of media operations (e.g. the coexisting number of video transcoding).
MediaMgmt.003	Media interface shall support obtaining the capacity provided by media acceleration resources.

5.12 QoS Functional group

5.12.1 Overview

A Quality of Service (QoS) system is widely used in network applications as to allow service providers to guarantee a service level agreement (SLA) to their subscribers and prevent oversubscription or abusive use of their network infrastructure. Due to its potential operational complexity and the impact this causes on latency and performance, offloading it to hardware is sought.

Figure 5.12.1-1 shows the sub-systems that exist in a QoS system in typical packet processing pipelines: Policing, Congestion Control, and Scheduling and Shaping respectively. Figure 5.12.1-2 shows the internal architecture of a typical multi-layer hierarchical policing sub-system. Figure 5.12.1-3 shows a typical congestion control system. Figure 5.12.1-4 shows a typical multi-layer hierarchical scheduling and shaping sub-system.

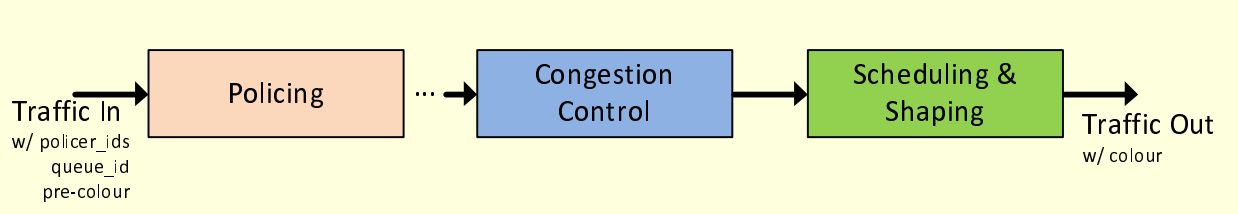


Figure 5.12.1-1: Typical QoS System

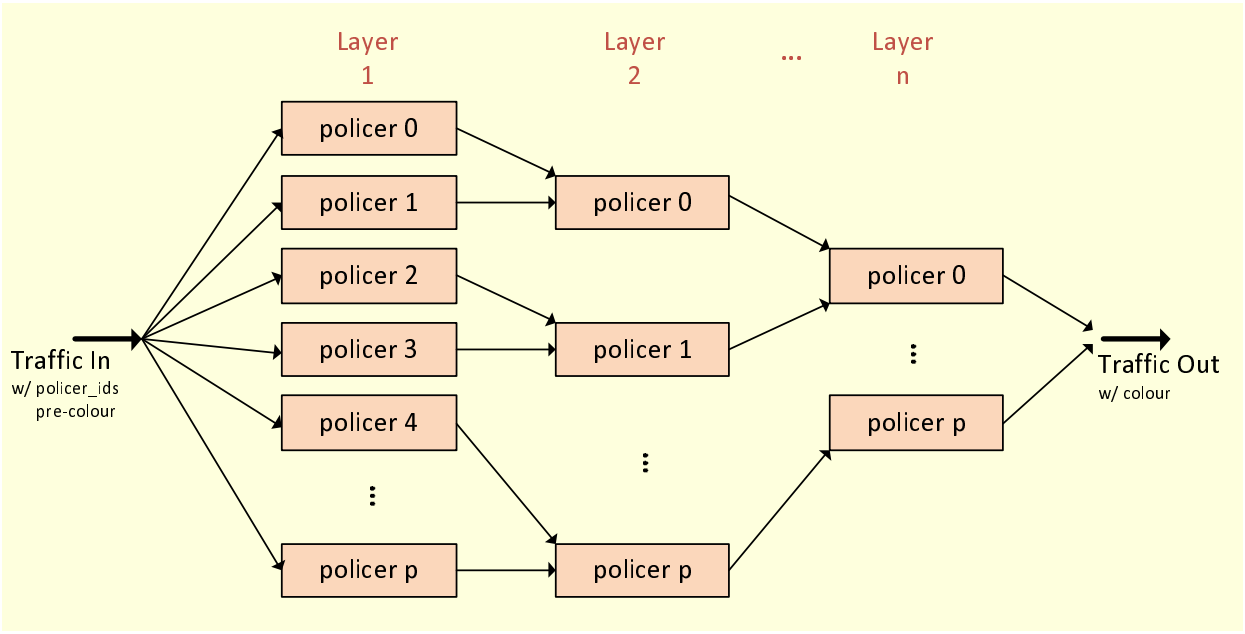


Figure 5.12.1-2: Typical Multi-Layers Hierarchical Policing System

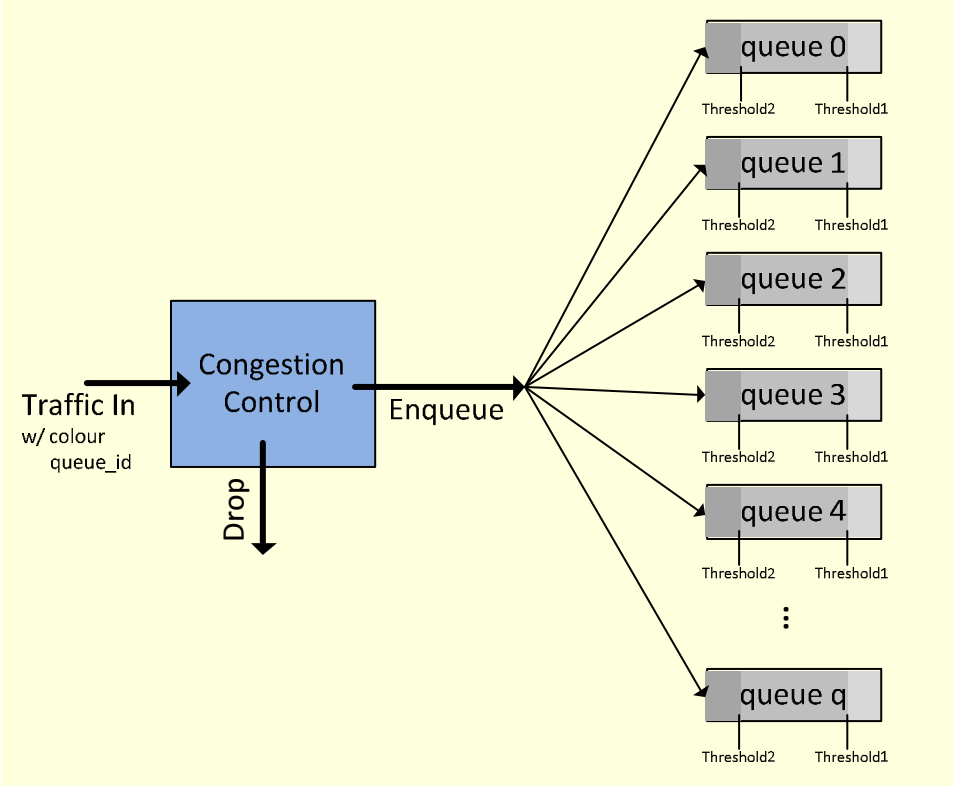


Figure 5.12.1-3: Typical Congestion Control System

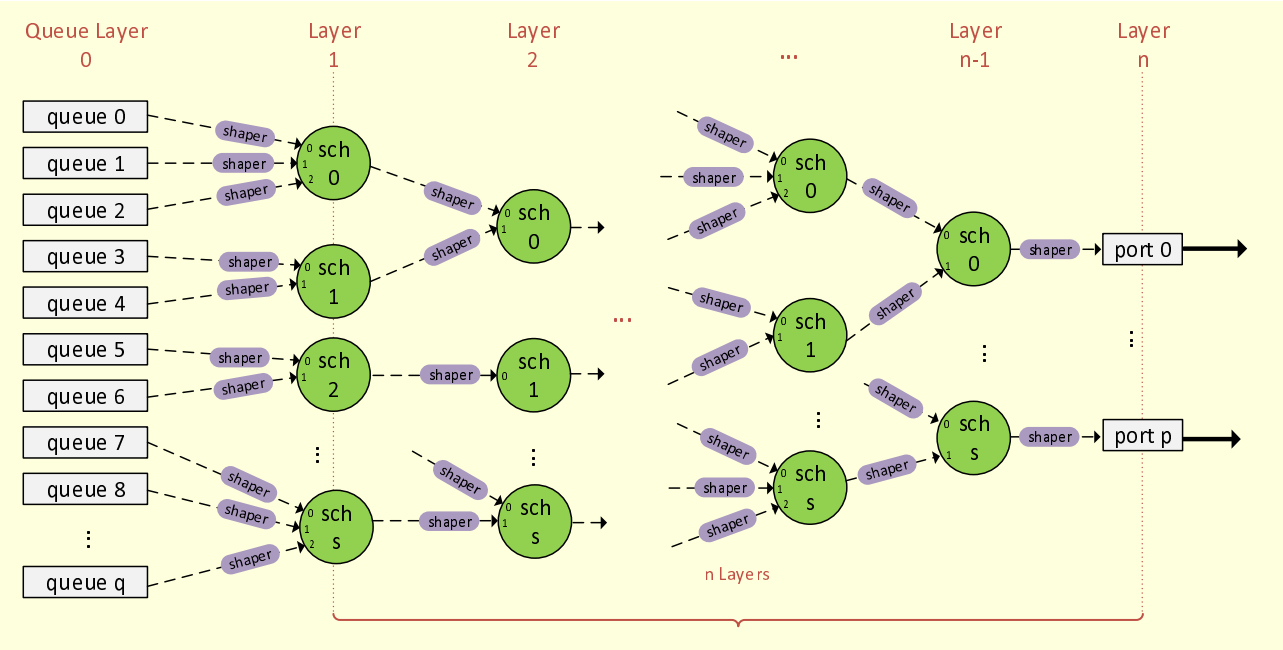


Figure 5.12.1-4: Typical Multi-Layer Hierarchical Scheduling and Shaping System

5.12.2 QoS interface requirements

Table 5.12.2-1

Numbering	Functional requirements description
VnNf.QoS.001	The QoS interface shall support obtaining the capabilities of a QoS system (Policing, Congestion Control, and Traffic Management and shaping capabilities).
VnNf.QoS.002	The QoS interface shall support initializing a QoS system.

5.12.3 QoS Operations requirements

Table 5.12.3-1

Numbering	Functional requirements description
QoS.Ops.001	The QoS interface shall support multi-layers policing system.
QoS.Ops.002	The QoS interface shall support one or more SLA profiles.
QoS.Ops.003	The QoS interface shall support setting/reading SLA profiles (CIR, PIR, CBS, EBS).
QoS.Ops.004	The QoS interface shall support mapping policers to SLA profiles.
QoS.Ops.005	The QoS interface shall support setting/reading congestion control algorithm.
QoS.Ops.006	The QoS interface shall support at least one threshold per congestion control queue.
QoS.Ops.007	The QoS interface shall support setting/reading queues thresholds.
QoS.Ops.008	The QoS interface shall support multi-layer scheduler.
QoS.Ops.009	The QoS interface shall support setting/reading scheduling parameters for each node in the scheduling hierarchy (such as scheduling algorithm, weight).
QoS.Ops.010	The QoS offloading interface shall support mapping any queue to any node in the highest layer.
QoS.Ops.011	The QoS interface shall support mapping any node in a particular layer to another node in the next lower layer.
QoS.Ops.012	The QoS interface shall support enabling/disabling shaping on a particular node leaf.
QoS.Ops.013	The QoS interface shall support setting/reading shaping profiles (such as CIR, PIR).
QoS.Ops.014	The QoS interface should support enqueuing traffic into a specific queue for look aside acceleration use cases.
QoS.Ops.015	The QoS interface should support dequeuing traffic for look aside acceleration use cases.

5.12.4 Management and monitoring requirements

Table 5.12.4-1

Numbering	Functional requirements description
QoS.Mgmt.001	The QoS interface shall support retrieving and resetting policing sub-system statistics.
QoS.Mgmt.002	The QoS interface shall support retrieving and resetting congestion control sub-system statistics.
QoS.Mgmt.003	The QoS interface shall support retrieving and resetting scheduling and shaping sub-system statistics.

5.13 PDCP Functional group

5.13.1 Overview

The following requirements apply to realize the Packet Data Convergence Protocol (PDCP) function.

NOTE: PDCP function is one of the compute-intensive processing for radio communication, the detailed introduction in ETSI TS 136 323 [2].

5.13.2 Overall requirements

Table 5.13.2-1

Numbering	Functional requirements description
PDCP.001	PDCP interface shall support acceleration of packet encryption and decryption processing with different security context.
PDCP.002	PDCP interface shall support acceleration of packet header compression.
PDCP.003	PDCP interface shall support acceleration of packet integrity protection.

5.13.3 Operations requirements

A PDCP accelerator should accelerate the packet processing for both the control plane and the data plane. For packets from the data plane, it should accelerate the packet header compression and packet encryption/decryption. For the packets from the control plane, it should accelerate the encryption/decryption and integrity protect.

Table 5.13.3-1

Numbering	Functional requirements description
PDCPOps.001	The PDCP interface should support applying queues to transmit different packet flow.
PDCPOps.002	The PDCP interface shall support transferring data to particular memory location.
PDCPOps.003	The PDCP interface shall support obtaining the configuration parameter for integrity protection.
PDCPOps.004	The PDCP interface shall support obtaining the configuration parameter for encryption and decryption.
PDCPOps.005	The PDCP interface shall support obtaining the configuration parameter for header compression.
PDCPOps.006	The PDCP interface shall support creating different security context per UE.
PDCPOps.007	The PDCP interface shall support retrieving security context.
PDCPOps.008	The PDCP interface shall support deleting security context. See note.
PDCPOps.009	The PDCP interface shall identify the different type of packet flow.
NOTE: The definition of security context can be found in the ETSI TS 133 401 [3].	

5.13.4 Management and monitoring requirements

Table 5.13.4-1

Numbering	Functional requirements description
PDCPmgmt.001	The PDCP interface shall support retrieving the status message from the accelerators. See note 2.
PDCPmgmt.002	The PDCP interface shall support managing the lifecycle of PDCP security context. See note 1.
NOTE 1: The definition of security context can be found in the ETSI TS 133 401 [3]. NOTE 2: The status message contains information that indicates the packet processing status, e.g. the sequence number of the packet that is currently processed.	

6 UML description of abstract interfaces

6.1 General

The Acceleration Model, as described in figure 6.1-1, is based on Extensible Para virtualised Devices (EPD) which implements one of more instance of Virtual Accelerators. Each driver of those Accelerators (EPD Frontend Driver) exposes an abstract interface of the corresponding Virtual Accelerator.

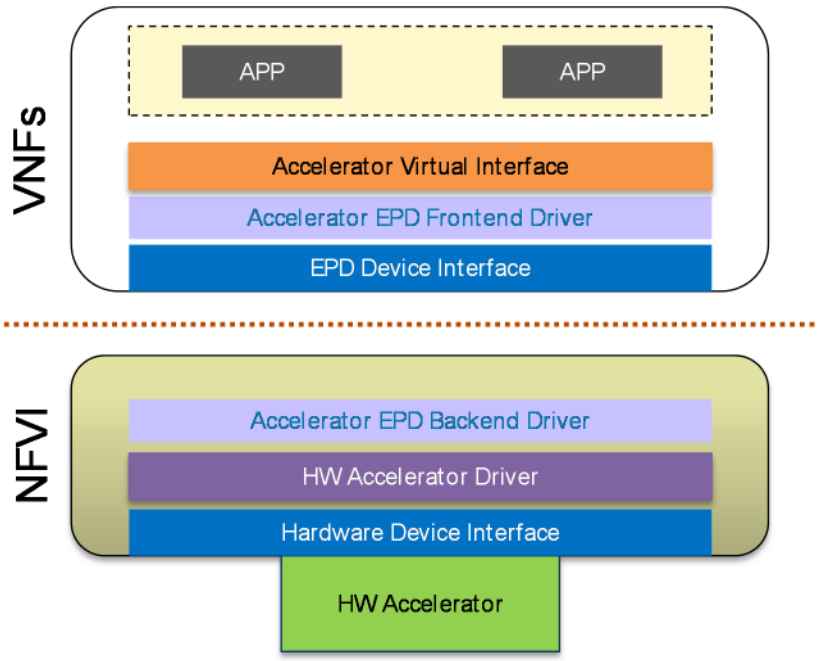


Figure 6.1-1: Acceleration model

6.2 Acceleration Virtualisation interface

6.2.1 Initialization

As mentioned earlier in the present document, Acceleration virtualisation interface is used to facilitate the communication between EPD Frontend and EPD Backend.

Before EPD Frontend can be used by VNF applications, it requires a plug-in from EPD Backend (which corresponds to the accelerator it is implementing) and also requires the accelerator to be initialized for the usage of that VNF application.

Figure 6.2.1-1 illustrates the typical information flow during the initialization phase.

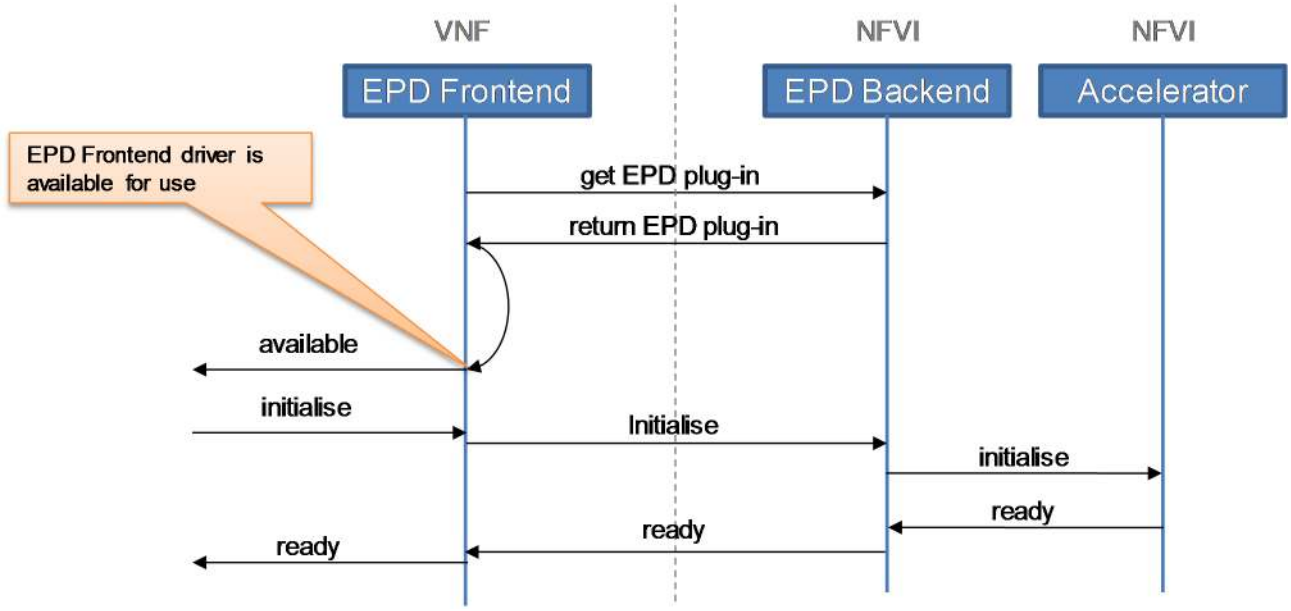


Figure 6.2.1-1: Information Flow during EPD initialization

The mechanism of which the EPD plug-in is injected into the VNF is beyond the scope of the present document.

6.2.2 Configuration

Acceleration Virtualisation Interface allows the EDP Frontend to configure the accelerator it has been given to. Figure 6.2.2-1 illustrates the information flow during the detection and the configuration of accelerator capabilities (such as number of channels, number of contexts, etc.).

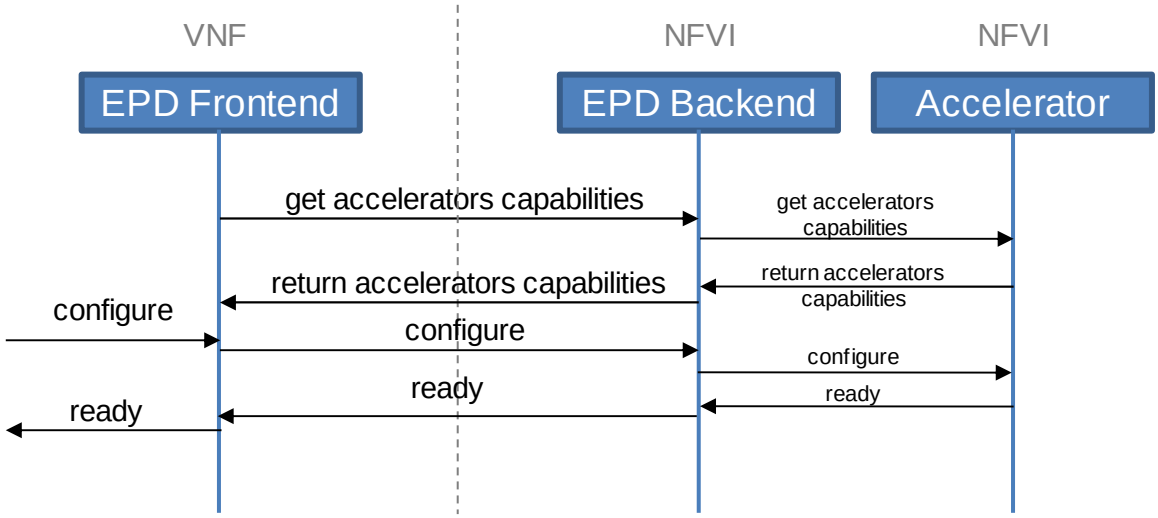


Figure 6.2.2-1: Information flow during configuration

6.2.3 Fault Management

In Scenarios when things go wrong while using acceleration, the VNF is expected to be able to detect the fault and be able to handle it. Clause 7.3 explains in more details the operations which Vn-Nf interface shall support in order to facilitate managing faults in accelerations. Figure 6.2.3-1 shows the typical information flow expected when handling faults in acceleration.

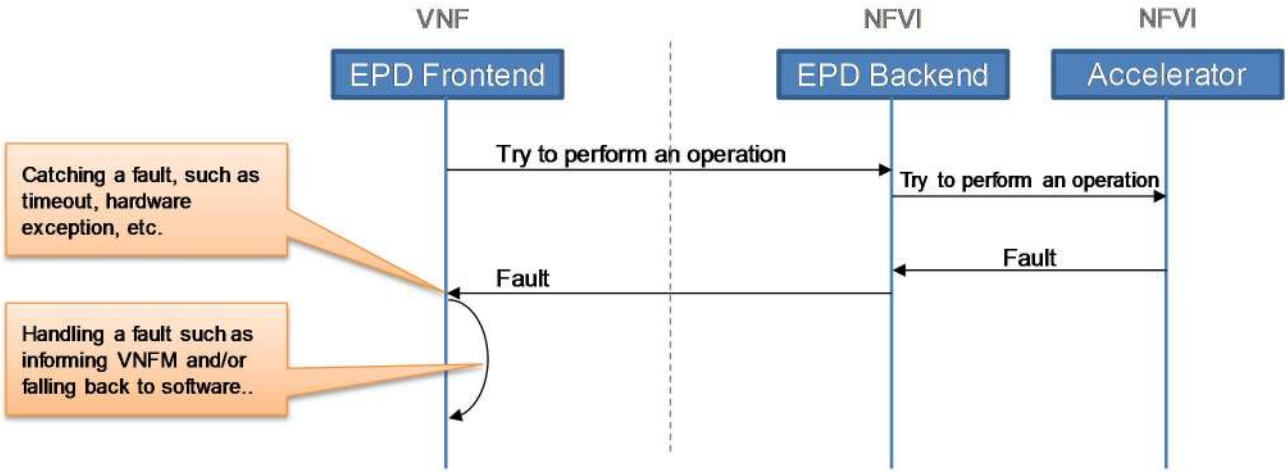


Figure 6.2.3-1: Information Flow during fault management

6.3 EPD Driver interface

6.3.1 Cryptography functional group

6.3.1.1 Overview

The cryptography functional group contains many functions, including key generation, packet encryption/decryption, hash, random number generation, etc. A crypto accelerator could support one or more of these functions, and a VNF could utilize these functions respectively or jointly. For example, the VNF may only request a hash operation to the accelerator, or may request both the key generation and key encryption/decryption functions for IPSec communication.

Thus the following clauses illustrate the flows of different cryptography functions respectively. When multiple functions are requested jointly, the "get accelerator's capabilities" and "initialize" processes are expected to be operated only once.

6.3.1.2 Key generation

This clause illustrates the general key generation process, shown in figure 6.3.1.2-1.

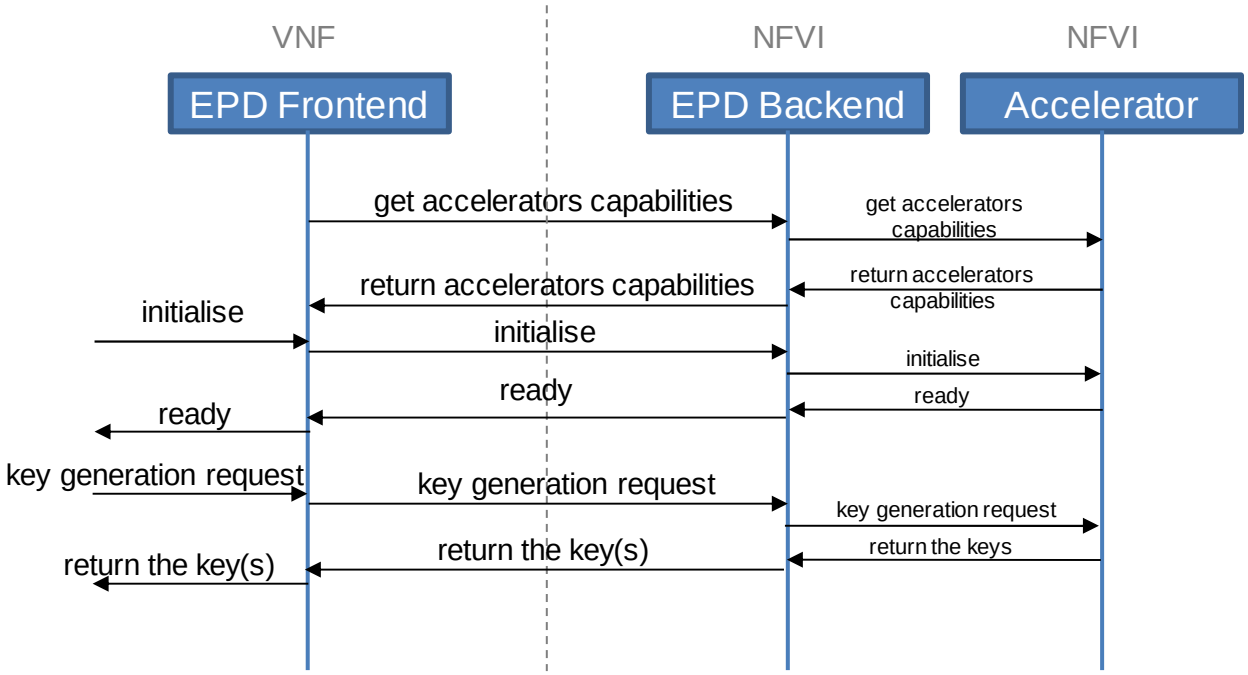


Figure 6.3.1.2-1: Key generation information flow

6.3.1.3 Operation execution

This clause illustrates the general operation execution process, which can be used for requesting crypto operations such as hash, packet encryption/decryption, etc.

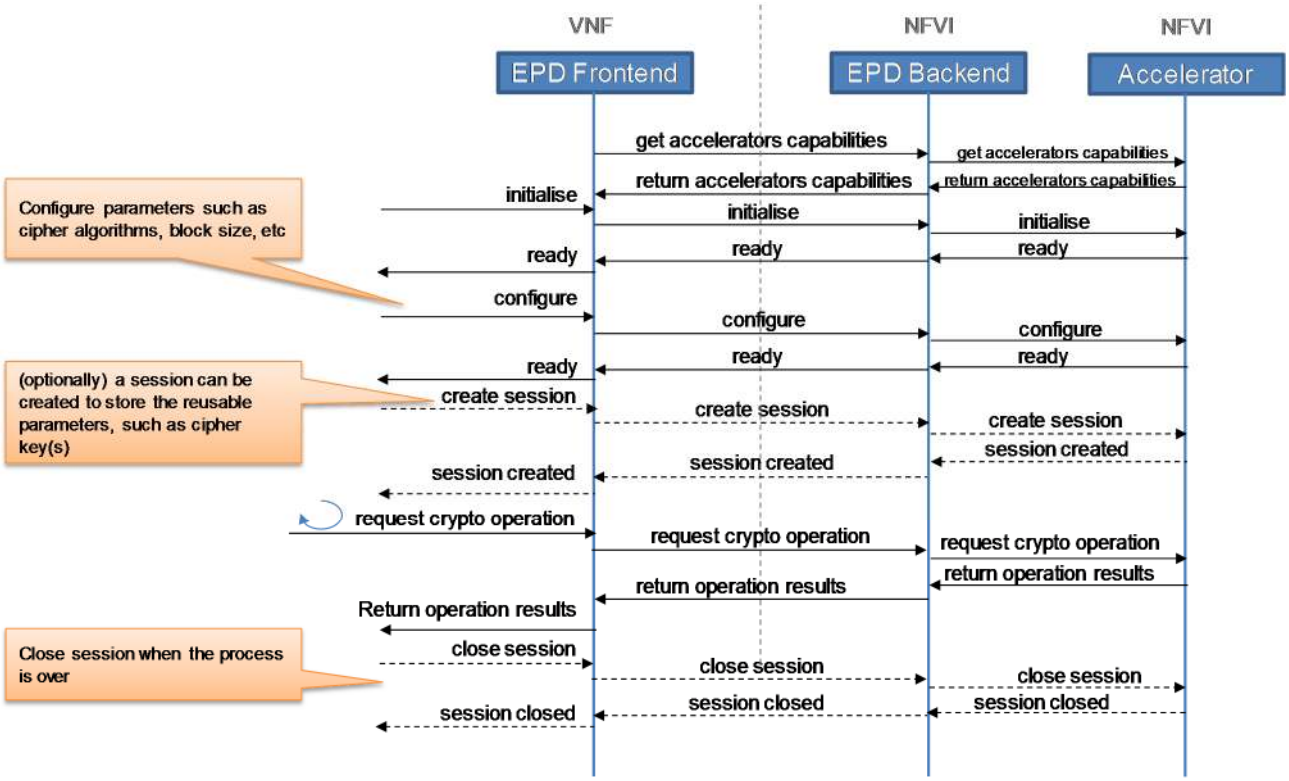


Figure 6.3.1.3-1: Operation execution information flow

6.3.1.4 Inline packet encryption/decryption

Besides look-aside operation illustrated in clause 6.3.1.3, the VNF could use an "inline" way to request the packet encryption/decryption operation. In the inline mode, the encryption/decryption process is combined with the packet transmission process, the accelerator is part of the transmission path, and the packet(s) are encrypted/decrypted as they traverse the accelerator. The VNF only sees the "raw" packets (i.e. the packets without encryption) in the inline mode. Figures 6.3.1.4-1 and 6.3.1.4-2 illustrate the information flow of the packet encryption/decryption in inline mode.

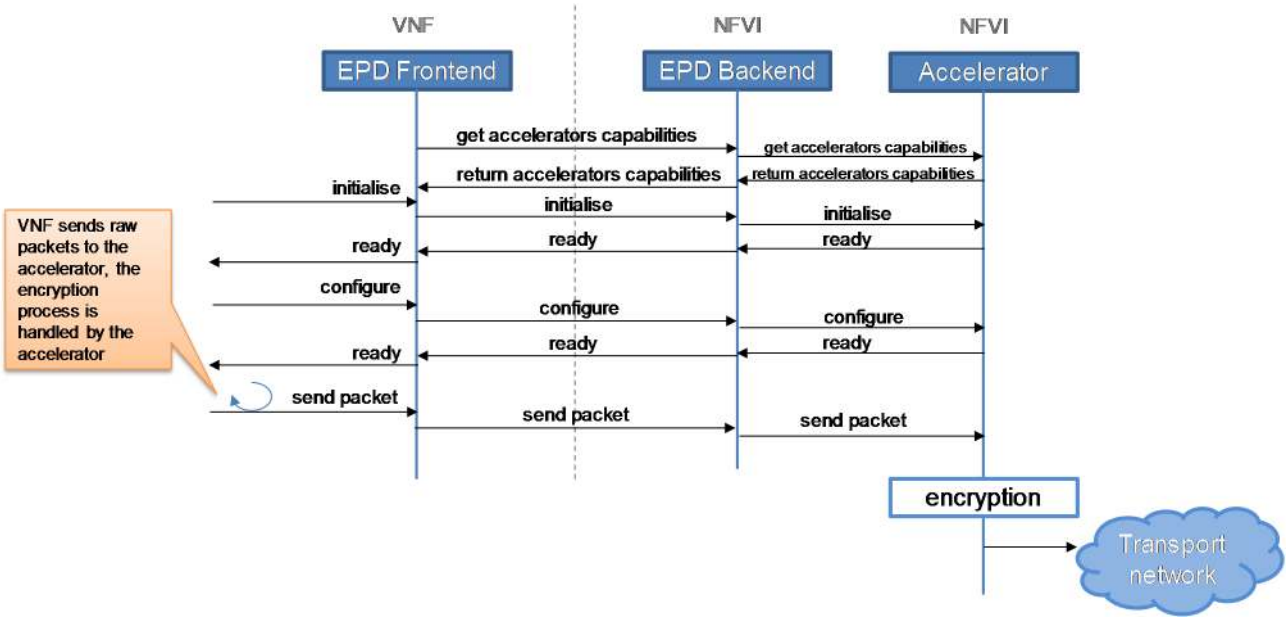


Figure 6.3.1.4-1: Inline packet encryption information flow

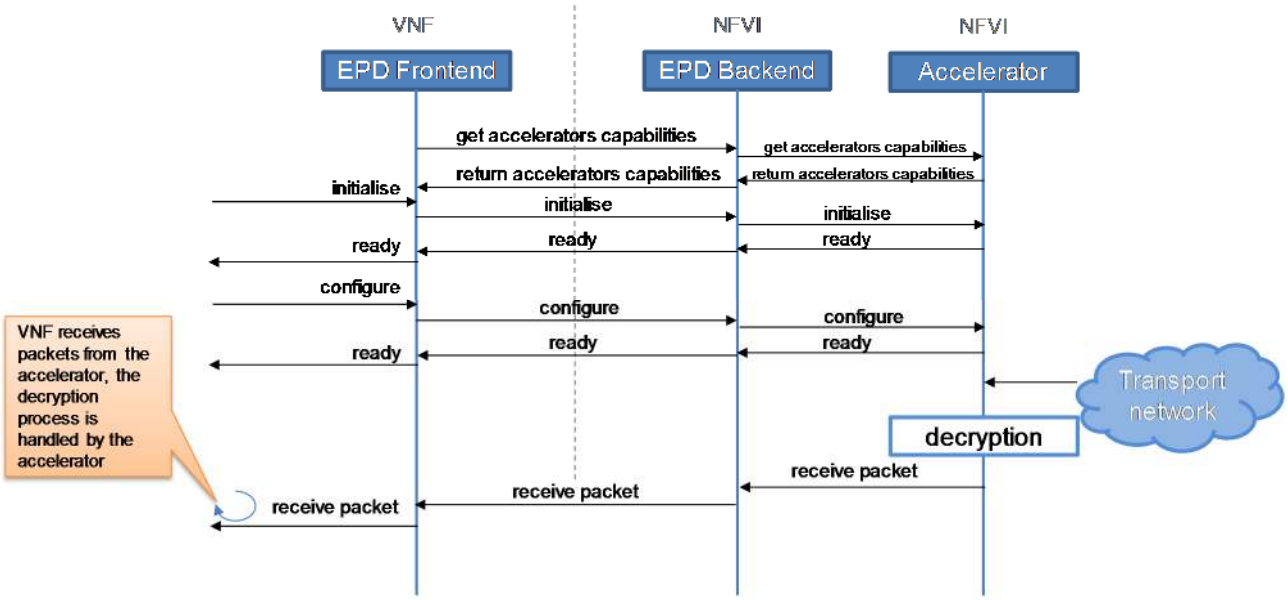


Figure 6.3.1.4-2: Inline packet decryption information flow

6.3.2 IPSec functional group

To meet the IPSec requirements listed in clause 5.5, IPSec VNF needs to discover the capability accelerator and configure it first, and then offloads the packets security processing to the accelerator who knows how to do that from the Security Association (SA) between the sender and receiver. Figures 6.3.2-1 and 6.3.2-2 illustrate the information flow of the IPSec packet encryption/decryption in inline mode.

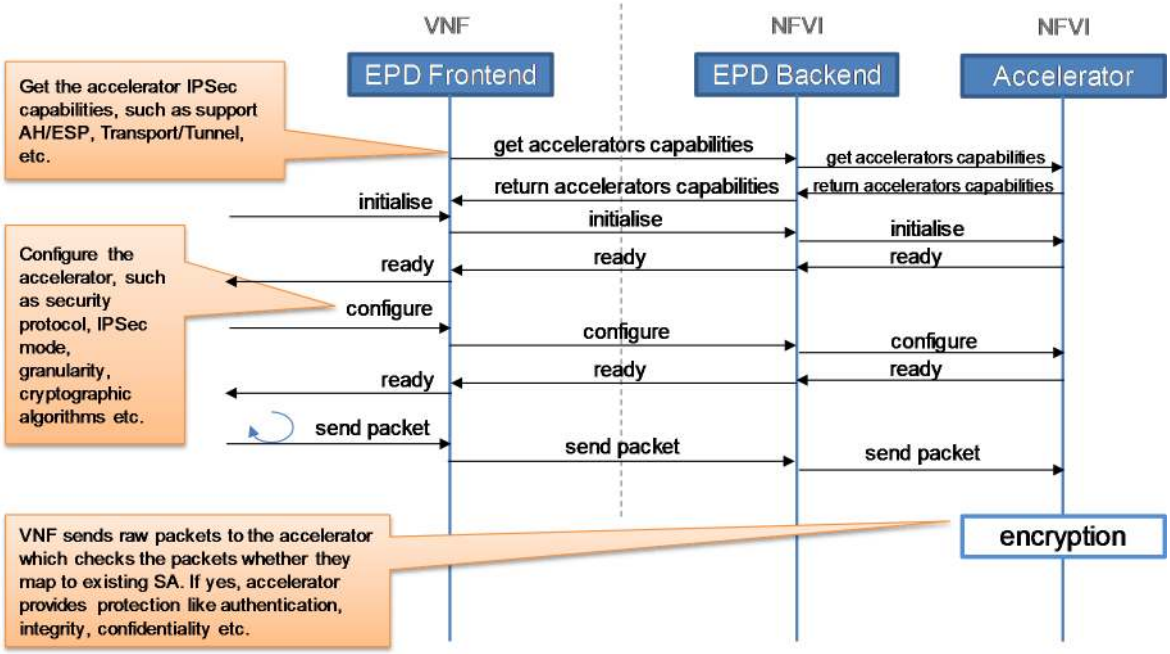


Figure 6.3.2-1: Packet encryption information flow

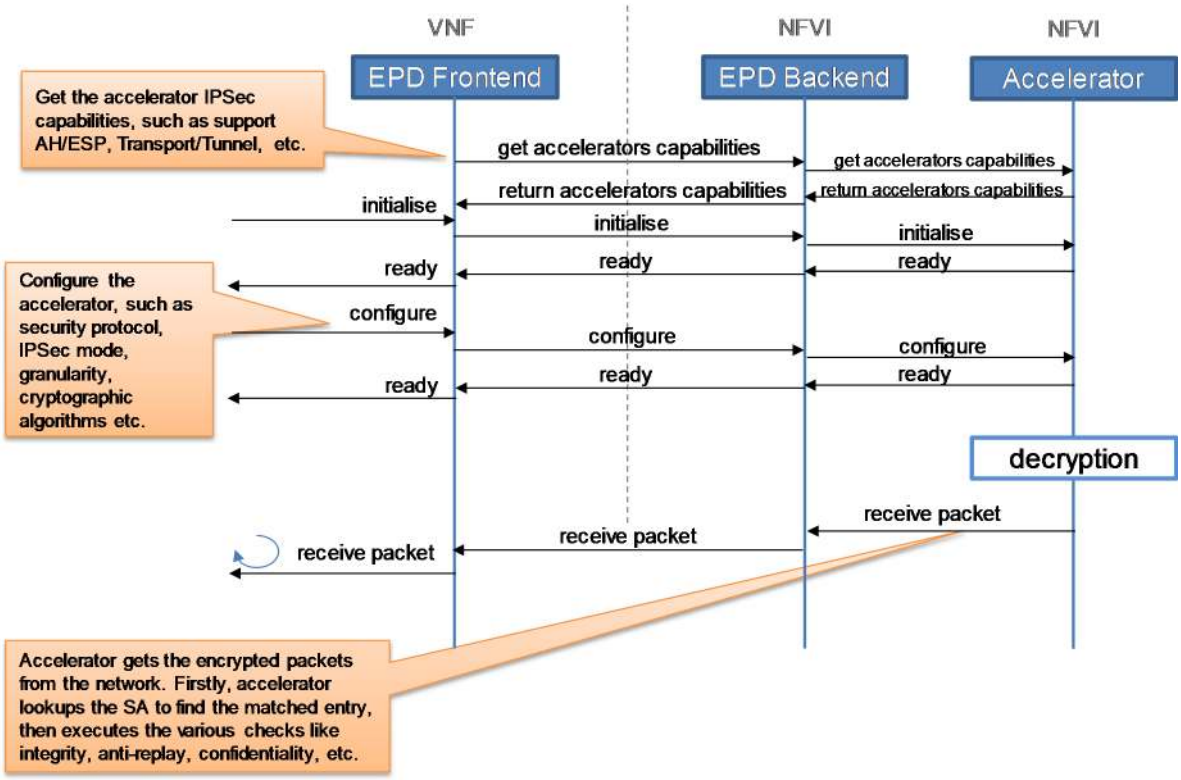


Figure 6.3.2-2: Packet decryption information flow

6.3.3 TCP functional group

To meet the TCP Interface requirements listed in clause 5.6, TCP VNF needs to discover the capability of the accelerator and configure it first, and then offloads some of the TCP/IP stack processing to the accelerator. Figure 6.3.3-1 illustrates the information flow of the TCP Interface.

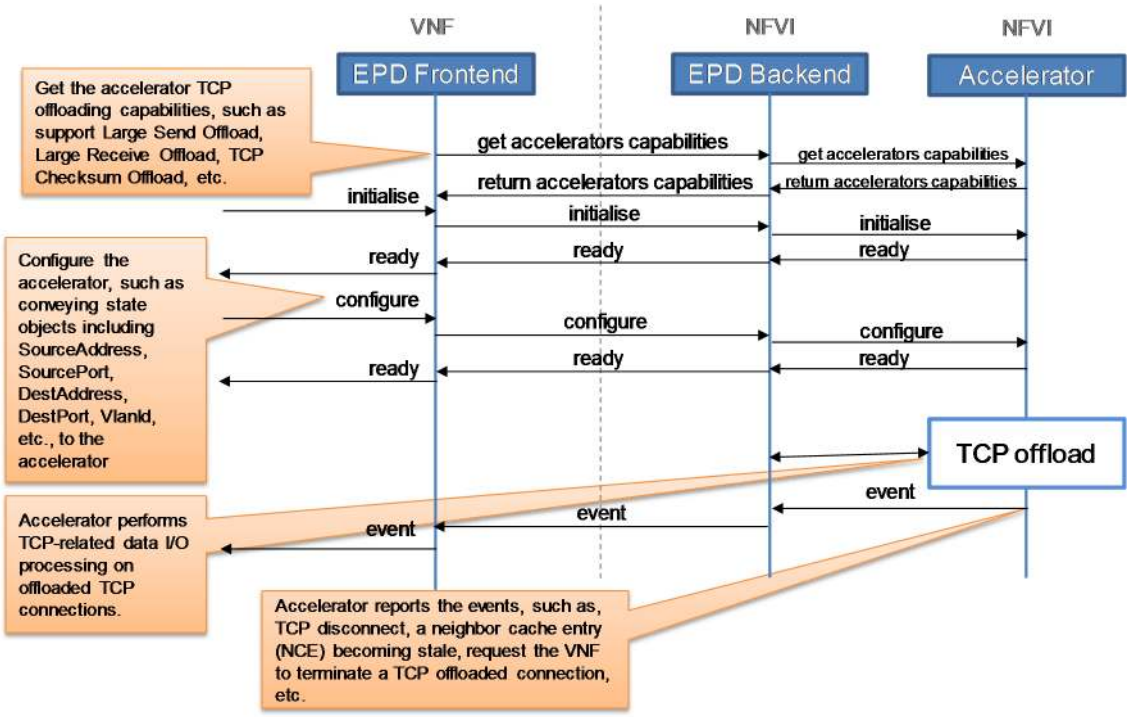


Figure 6.3.3-1: TCP Interface information flow

6.3.4 Re-programmable computing functional group

The information flow that is specific to Re-programmable computing functional group accelerators is illustrated in figure 6.3.4-1.

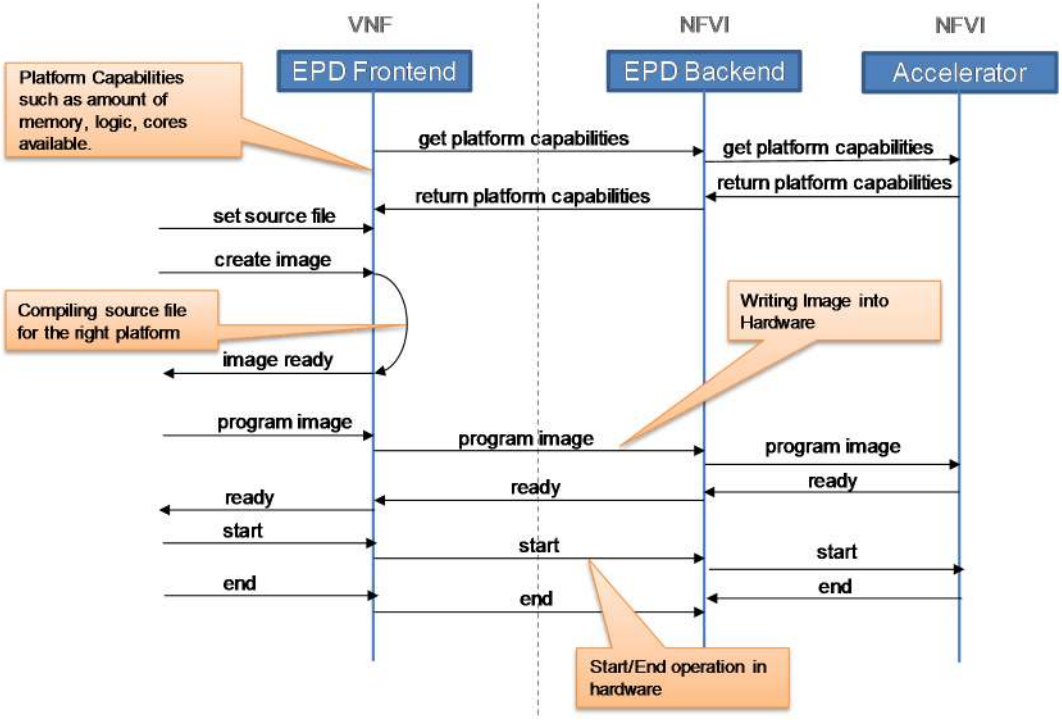


Figure 6.3.4-1: Re-Programmable computing functional group information flow

6.3.5 Dynamic Optimization of Packet Flow Routing Functional Group

To meet the DOPFR requirements listed in clause 5.9, VNF needs to discover the capability of the accelerator and configure it first, and then offloads some or all of data plane processing into the accelerator. Figure 6.3.5-1 illustrates the information flow of the DOPFR interface.

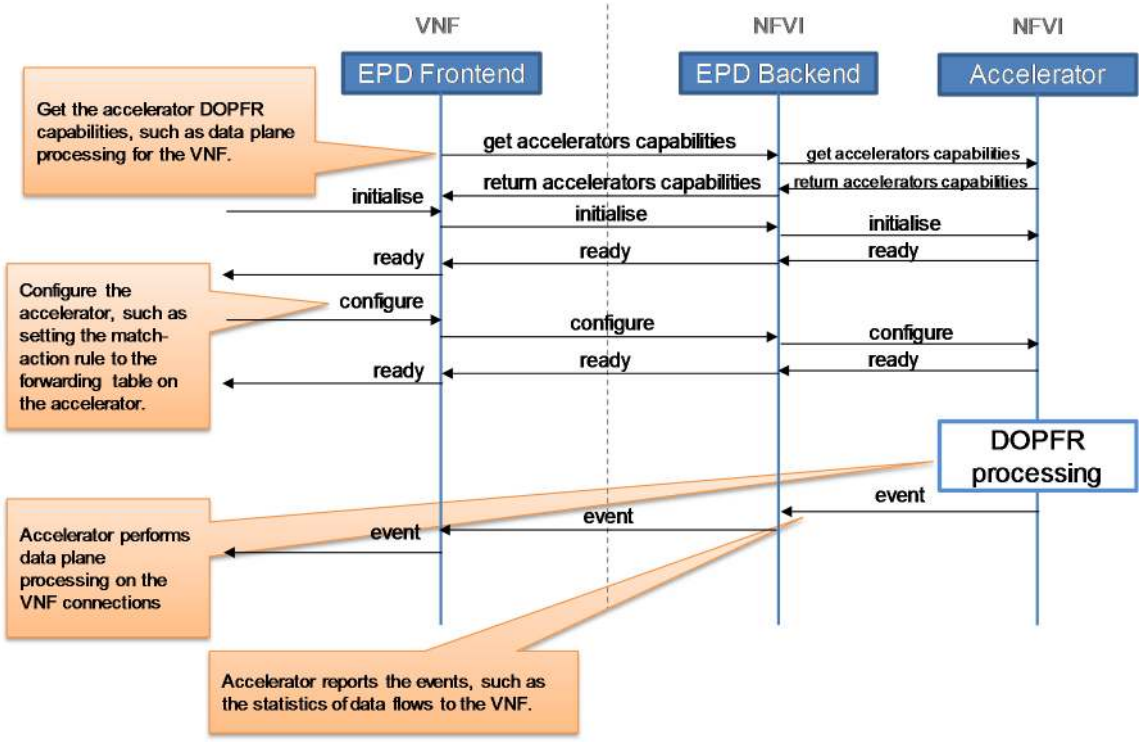


Figure 6.3.5-1: DOPRF information flow

6.3.6 NAT functional group

The information flow that is specific to NAT offloading functional group acceleration use case is illustrated in figure 6.3.6-1.

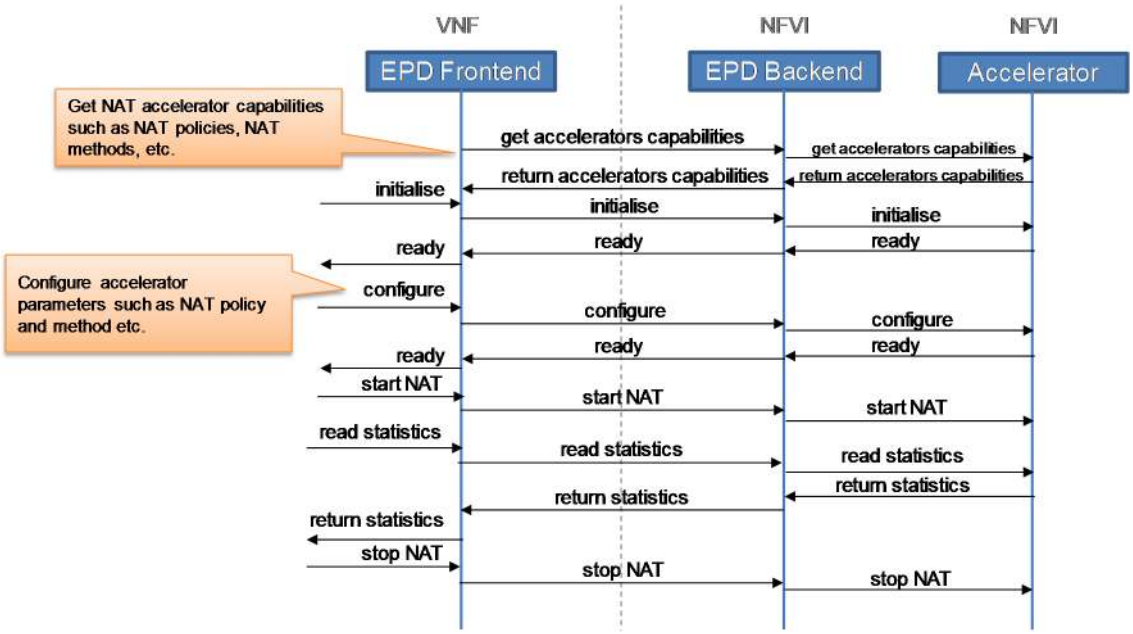


Figure 6.3.6-1: NAT offloading functional group information flow

6.3.7 VXLAN functional group

The information flow that is specific to VXLAN functional group accelerators is illustrated in figure 6.3.7-1.

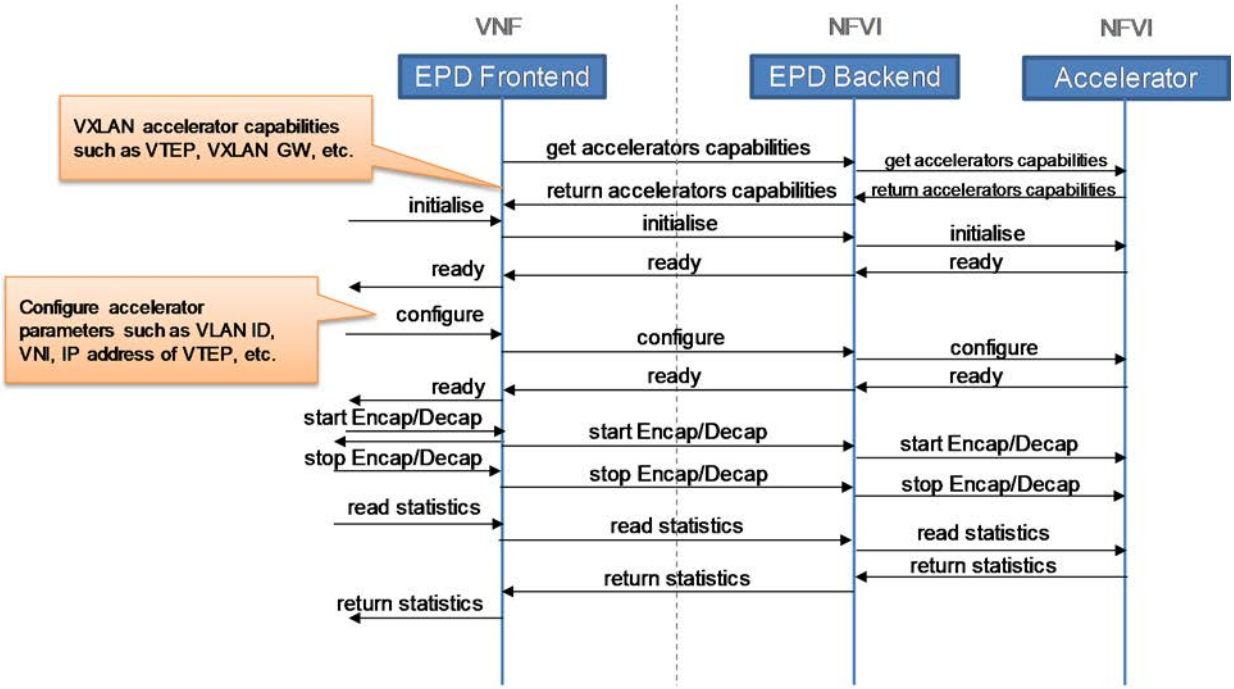


Figure 6.3-7-1: VXLAN functional group information flow

6.3.8 Media functional group

According to clause 5.12, the media accelerators shall support basic media acceleration operations, including: transcoding, encoding/decoding and mixing for audio and/or video. Figures 6.3.8-1 to 6.3.8-4 illustrate the flows of these operations.

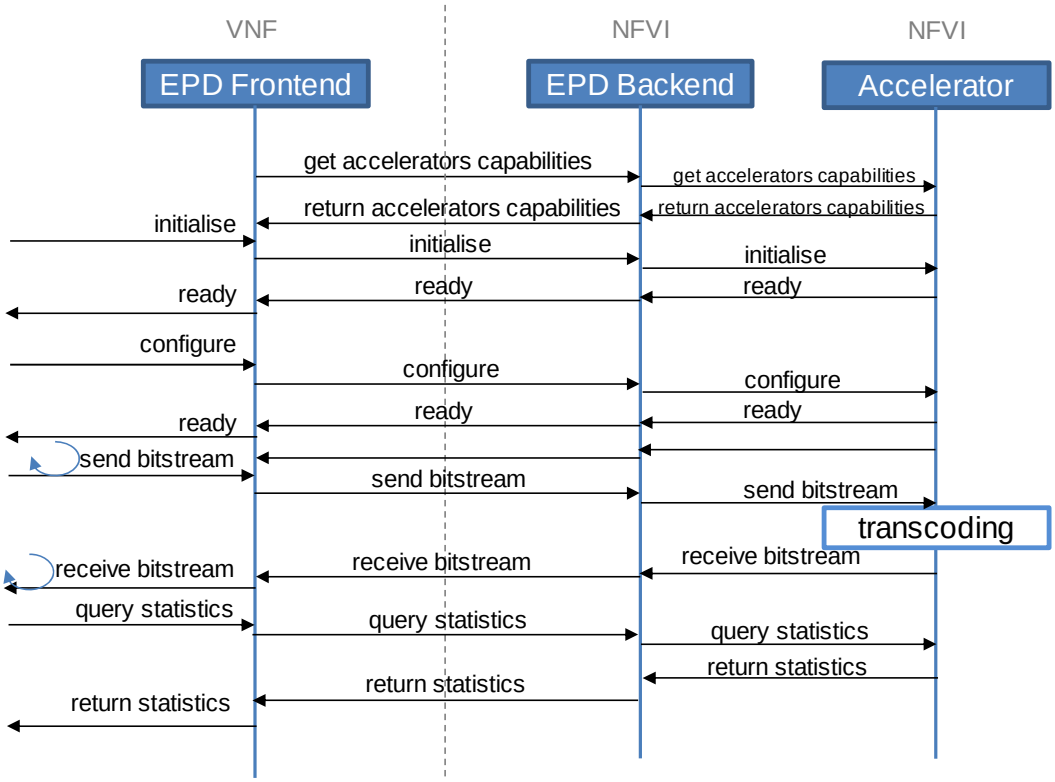


Figure 6.3.8-1: Media transcoding information flow

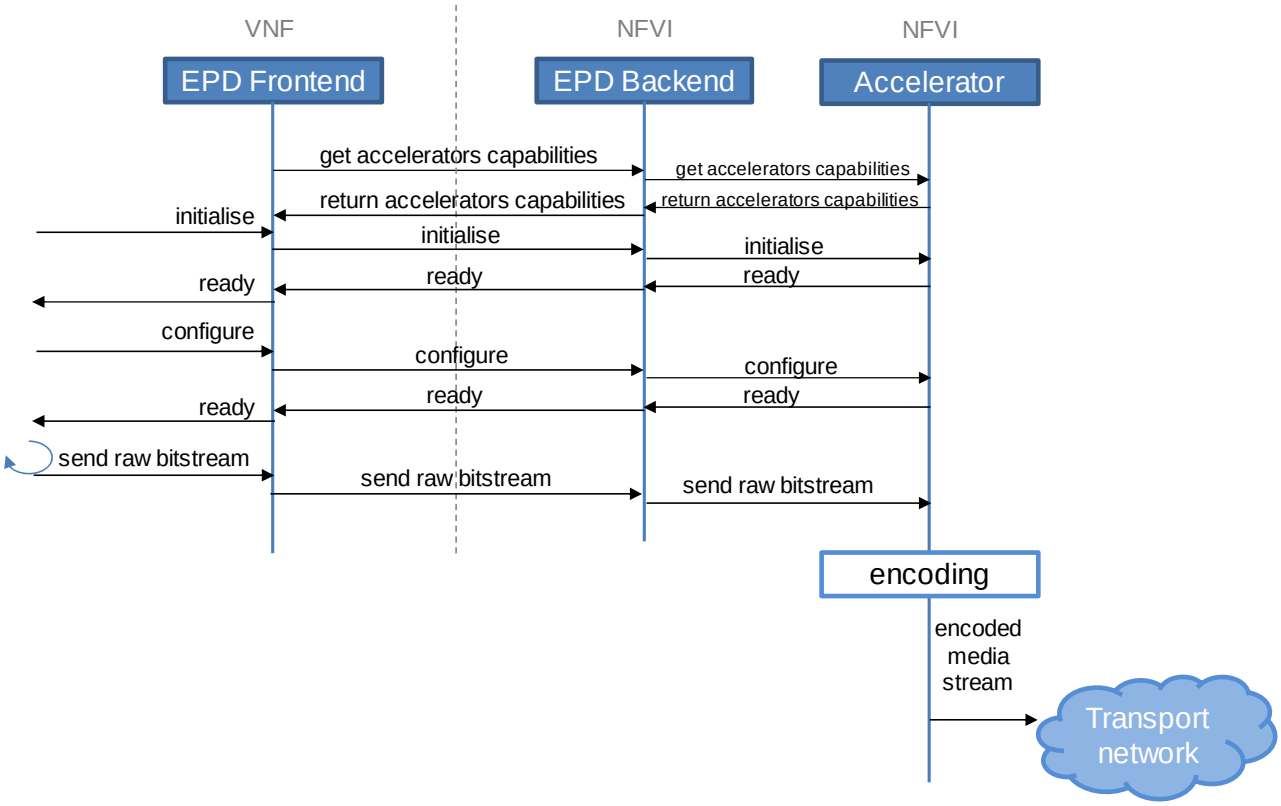


Figure 6.3.8-2: Media encoding information flow

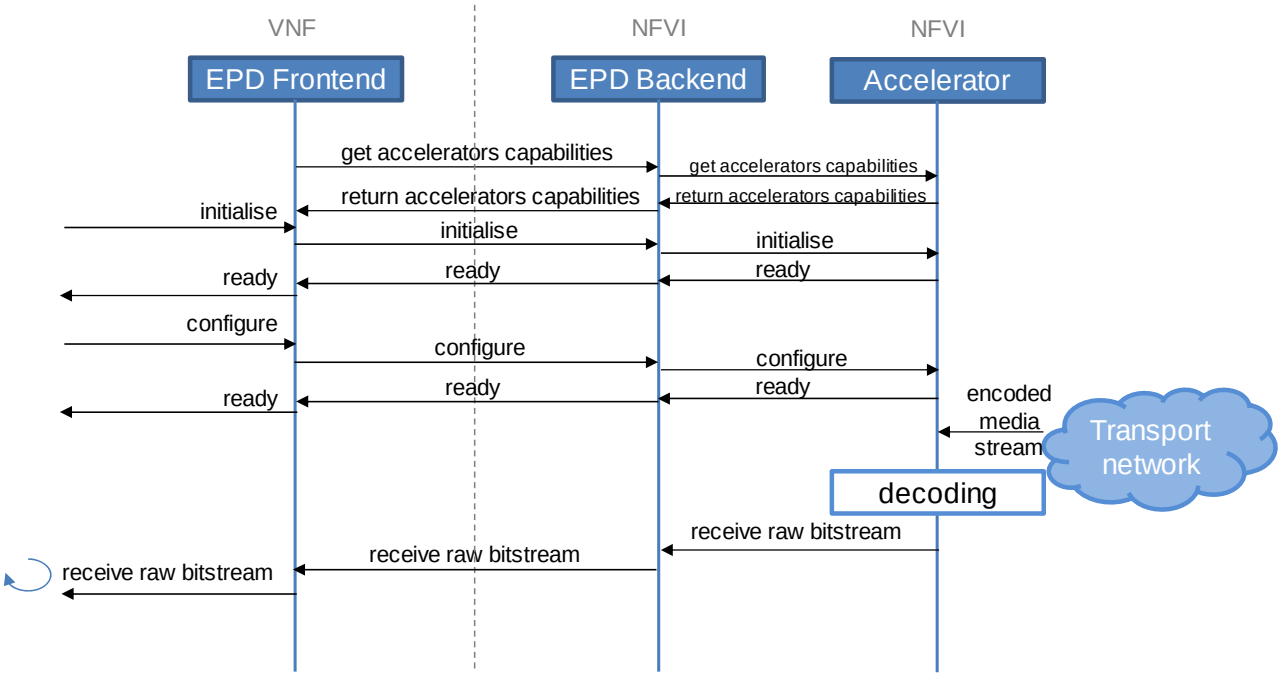


Figure 6.3.8-3: Media decoding information flow

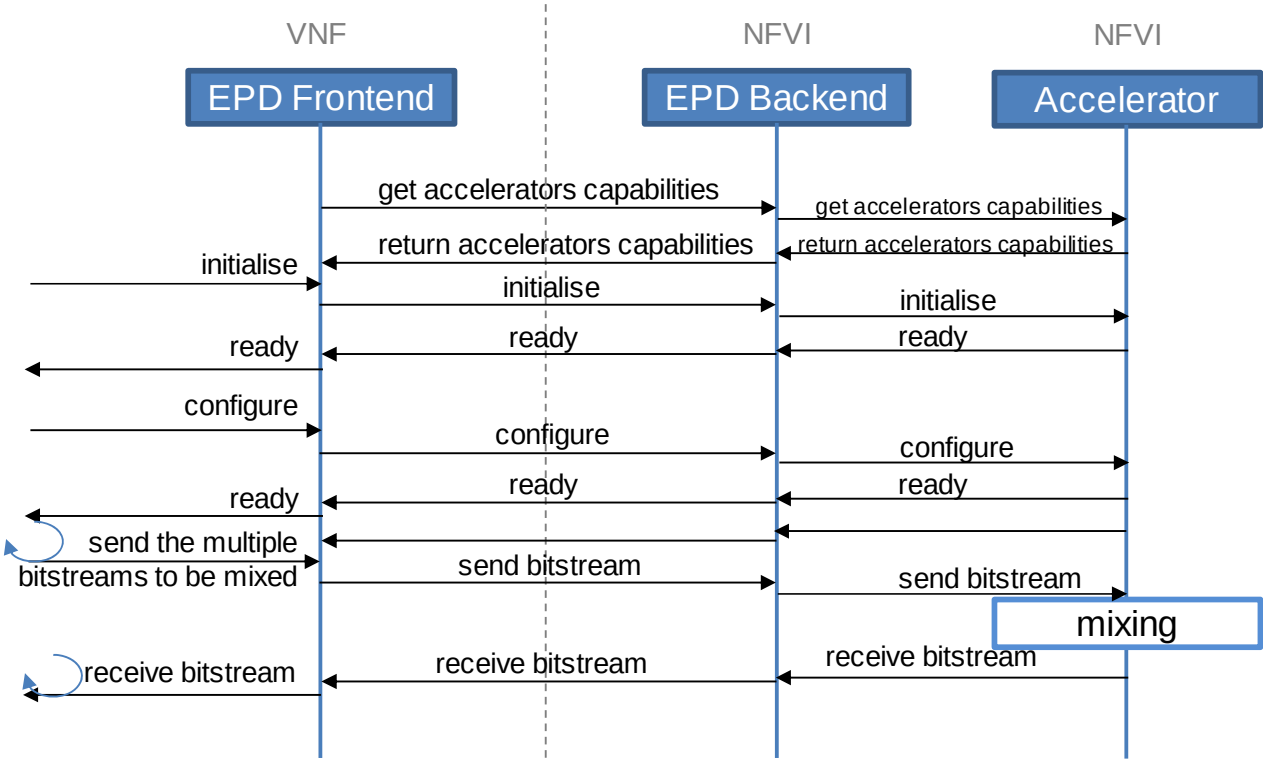


Figure 6.3.8-4: Media mixing information flow

6.3.9 QoS Functional group

As seen in clause 5.12, a QoS system consists of different sub-systems namely: policing, congestion control and scheduling and shaping. Before VNFs can use an offloaded QoS system, they need to discover it and configure it first. Figure 6.3.9-1 shows the sequence of information flow by VNFs accessing an accelerated QoS system.

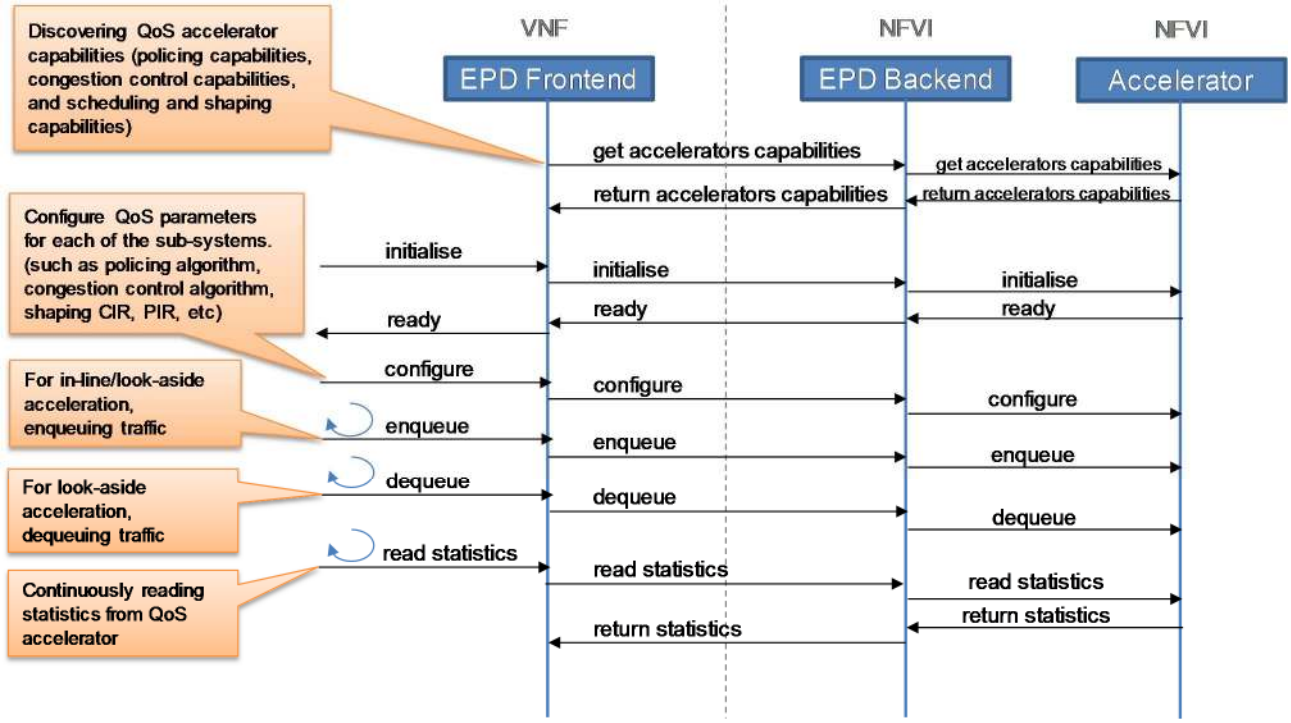


Figure 6.3.9-1: QoS Interface information flow

7 VNF Interfaces Specifications

7.1 General

This clause defines interfaces exposed by the NFVI towards VNFs and interfaces exposed by VNFs towards NFVI over the Vn-Nf reference point.

Each of the acceleration functional group will have its own VNF interface whihch implements some or all of the operations described in this clause.

NOTE: The fact that information elements and attributes are presented in tabular form does not preclude stage 3 designs in which these information elements and attributes are encoded in different parts of request and response messages. For example, in a RESTful interface, parts of them can be encoded in the URL, in the message header, in the message body or any combination thereof.

7.2 Conventions

The following notations, defined in ISO/IEC 9646-7 [i.8], are used for the qualifier column of interface information elements:

- M mandatory - the capability is required to be supported.
- O optional - the capability may be supported or not.
- N/A not applicable - in the given context, it is impossible to use the capability.
- CM conditional mandatory - the capability is required to be supported and is conditional on the support of some condition. This condition shall be specified in the Description column.
- CO conditional optional - the capability may be supported or not and is conditional on the support of some condition. This condition shall be specified in the Description column.

7.3 EPD Management interface

7.3.1 Description

This interface allows the VNF to query information about the acceleration capabilities from the NFVI. It also includes subscribe and notify operations for acceleration capabilities changes.

7.3.2 Init operation

7.3.2.1 Description

The init operation initializes the accelerator and retrieves its capabilities from the NFVI. Table 7.3.2.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.3.2.1-1: Init Accelerator Operation

Message	Requirement	Direction
InitAccRequest	Mandatory	VNF → NFVI
InitAccResponse	Mandatory	NFVI → VNF

7.3.2.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.3.2.2-1.

Table 7.3.2.2-1: Init operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accFilter	M	1	Filter	This is used to specify one or more sub-system(s) within the accelerator to initialize them and retrieve their capabilities.
accAttributeSelector	M	0..N	enum	It provides a list of attribute names of AccCapabilities. If present, only these attributes will be returned for the accelerator sub-system(s) matching the filter. If absent, the complete capability list of the chosen sub-system will be returned.

7.3.2.3 Output Parameters

The output parameters returned by the operation shall follow the indications provided in table 7.3.2.3-1.

Table 7.3.2.3-1: Init operation output parameters

Parameter	Qualifier	Cardinality	Content	Description
accCapabilities	M	0..N	AccCapabilities	Details of the on-boarded acceleration sub-system capabilities matching the input filter. If accAttributeSelector is present, only the attributes listed in accAttributeSelector will be returned for the selected sub-systems.

7.3.3 RegisterForAccEvent operation

7.3.3.1 Description

The RegisterForAccEvent operation allows the VNF to be informed about any event that is supported by accelerator (such as availability of data, exception, etc.). Table 7.3.3.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.3.3.1-1: RegisterForAccEvent Operation

Message	Requirement	Direction
RegisterForAccEventRequest	Mandatory	VNF → NFVI
RegisterForAccEventRequest	Mandatory	NFVI → VNF

7.3.3.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.3.3.2-1.

Table 7.3.3.2-1: RegisterForAccEvent operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accEvent	M	1	AccEventType	Identify what event the VNF is interested in.
vnfEventHandlerId	M	1	Identifier	Identifies the handler (reference) for NFVI to use when notifying the VNF about the event.

7.3.3.3 Output Parameters

None.

7.3.3.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the VNF has been successfully registered for the event specified by accEvent.

7.3.4 AccEventNotification operation

7.3.4.1 Description

The AccEventNotification operation allows the NFVI to notify VNF about events it has registered to. Table 7.3.4.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.3.4.1-1: AccEventNotification Operation

Message	Requirement	Direction
AccEventNotificationRequest	Mandatory	NFVI → VNF
AccEventNotificationResponse	Mandatory	VNF → NFVI

7.3.4.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.3.4.2-1.

Table 7.3.4.2-1: AccEventNotification operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
vnfEventHandlerId	M	1	Identifier	This is reference used by VNF when registering for a particular event.
accEventMetaData	M	0 … N	AccEventMetaData	Any Metadata that comes with an event.

7.3.4.3 Output Parameters

None.

7.3.4.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the VNF has been successfully notified about the relevant event.

7.3.5 DeRegisterForAccEvent operation

7.3.5.1 Description

The DeRegisterForAccEvent operation allows the VNF to be De-Registered from any event it has previously registered to. Table 7.3.5.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.3.5.1-1: DeRegisterForAccEvent Operation

Message	Requirement	Direction
DeRegisterForAccEventRequest	Mandatory	VNF → NFVI
DeRegisterForAccEventRequest	Mandatory	NFVI → VNF

7.3.5.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.3.5.2-1.

Table 7.3.5.2-1: DeRegisterForAccEvent operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accEvent	M	1	AccEventType	Identify what event the VNF is interested in.

7.3.5.3 Output Parameters

None.

7.3.5.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the VNF has been successfully de-registered from the event specified by accEvent.

7.3.6 ReleaseAcc operation

7.3.6.1 Description

The operation allows VNF to request a release from an accelerator. Table 7.3.6.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.3.6.1-1: ReleaseAcc Operation

Message	Requirement	Direction
ReleaseAccRequest	Mandatory	VNF → NFVI
ReleaseAccResponse	Mandatory	NFVI → VNF

7.3.6.2 Input Parameters

None.

7.3.6.3 Output Parameters

None.

7.3.6.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the VNF is no longer has an access to the accelerator.

7.4 EPD Configuration Interface

7.4.1 Description

This interface allows the VNF to provide configuration information for an accelerator, or an individual sub-system within that accelerator.

NOTE: This clause only covers the case when VNFs and NFVI are in the same domain.

7.4.2 ModifyAccConfiguration operation

7.4.2.1 Description

This operation enables providing the configuration parameters of an accelerator and its sub-system(s) or individual sub-system.

Table 7.4.2.1-1 lists the information flow exchanged between the VNF and the NFVI.

Table 7.4.2.1-1: ModifyAccConfiguration Operation

Message	Requirement	Direction
ModifyAccConfigurationRequest	Mandatory	VNF → NFVI
ModifyAccConfigurationResponse	Mandatory	NFVI → VNF

7.4.2.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.4.2.2-1.

Table 7.4.2.2-1: ModifyAccConfiguration operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accConfigurationData	M	0..1	AccConfigurationType	Configuration data for the Accelerator (see note 1 and note 2).
accSubSysConfigurationData	M	0..N	AccSubSysConfigurationType	Configuration data for a particular sub-system (see note 1 and note 3).
NOTE 1: At least one accConfigurationData or accSubSysConfigurationData element shall be included. NOTE 2: Cardinality of 0 is used when the operation is used for configuration of only individual sub-system. NOTE 3: Cardinality of 0 is used when the operation is used for configuration of an accelerator and all its sub-systems.				

7.4.2.3 Output Parameters

None.

7.4.2.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the configuration in the accelerator has been modified according to the input parameters specified in the operation.

7.4.3 GetAccConfigs operation

7.4.3.1 Description

The operation retrieves configuration from accelerator's functions. Table 7.4.3.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.4.3.1-1: GetAccConfigs Operation

Message	Requirement	Direction
GetAccConfigsRequest	Mandatory	VNF → NFVI
GetAccConfigsResponse	Mandatory	NFVI → VNF

7.4.3.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.4.3.2-1.

Table 7.4.3.2-1: GetAccConfigs input parameters

Parameter	Qualifier	Cardinality	Content	Description
accFilter	M	1	Filter	This is used to specify one or more sub-system(s) within the accelerator from which the configuration is requested from.
accConfigSelector	M	0..N	enum	It provides a list of attribute names of AccConfigurationType. If present, only these attributes will be returned for the accelerator sub-system(s) matching the filter. If absent, the complete Configuration list of the chosen sub-system will be returned.

7.4.3.3 Output Parameters

The output parameters returned by the operation shall follow the indications provided in table 7.4.3.3-1.

Table 7.4.3.3-1: GetAccConfigs output parameters

Parameter	Qualifier	Cardinality	Content	Description
accConfigs	M	0..N	AccConfigurationType	Configuration of the on-boarded acceleration sub-system matching the input filter. If accConfigSelector is present, only the attributes listed in accConfigSelector will be returned for the selected sub-systems.

7.4.3.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the configuration of the accelerator has been returned according to the input parameters specified in the operation.

7.4.4 ResetAccConfigs operation

7.4.4.1 Description

The operation resets the configuration of a given accelerator's functions. Table 7.4.4.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.4.4.1-1: ResetAccConfigs Operation

Message	Requirement	Direction
ResetAccConfigsRequest	Mandatory	VNF → NFVI
ResetAccConfigsResponse	Mandatory	NFVI → VNF

7.4.4.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.4.4.2-1.

Table 7.4.4.2-1: ResetAccConfigs input parameters

Parameter	Qualifier	Cardinality	Content	Description
accFilter	M	1	Filter	This is used to specify one or more sub-system(s) within the accelerator for which the configuration is to be reset.
accConfigSelector	M	0..N	enum	It provides a list of attribute names of AccConfigurationType. If present, only configurations related to these attributes will be reset for the accelerator sub-system(s) matching the filter. If absent, all the related configurations of the chosen sub-system will be reset.

7.4.4.3 Output Parameters

None.

7.4.4.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the resetting of the configuration has been completed according to the input parameters specified in the operation.

7.5 EPD Monitoring Interface

7.5.1 Description

This interface allows the VNF to get monitoring information from the accelerator's (or its sub-system(s)'s) functions.

7.5.2 GetAccStatistics operation

7.5.2.1 Description

The operation retrieves statistics from accelerator's functions. Table 7.5.2.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.5.2.1-1: GetAccStatistics Operation

Message	Requirement	Direction
GetAccStatisticsRequest	Mandatory	VNF → NFVI
GetAccStatisticsResponse	Mandatory	NFVI → VNF

7.5.2.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.5.2.2-1.

Table 7.5.2.2-1: GetAccStatistics input parameters

Parameter	Qualifier	Cardinality	Content	Description
accFilter	M	1	Filter	This is used to specify one or more sub-system(s) within the accelerator from which the statistical data is requested from.
accStatSelector	M	0..N	enum	It provides a list of attribute names of AccStatistics. If present, only these attributes will be returned for the accelerator sub-system(s) matching the filter. If absent, the complete statistics list of the chosen sub-system will be returned.

7.5.2.3 Output Parameters

The output parameters returned by the operation shall follow the indications provided in table 7.5.2.3-1.

Table 7.5.2.3-1: GetAccStatistics output parameters

Parameter	Qualifier	Cardinality	Content	Description
accStatistics	M	0..N	AccStatistics	Statistical data of the on-boarded acceleration sub-system matching the input filter. If accStatSelector is present, only the attributes listed in accStatSelector will be returned for the selected sub-systems.

7.5.2.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the resetting of the statistical information in the accelerator has been returned according to the input parameters specified in the operation.

7.5.3 ResetAccStatistics operation

7.5.3.1 Description

The operation resets the statistics for a given accelerator's functions. Table 7.5.3.1-1 lists the information exchanged between VNFs and the NFVI.

Table 7.5.3.1-1: ResetAccStatistics Operation

Message	Requirement	Direction
ResetAccStatisticsRequest	Mandatory	VNF → NFVI
ResetAccStatisticsResponse	Mandatory	NFVI → VNF

7.5.3.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.5.3.2-1.

Table 7.5.3.2-1: ResetAccStatistics input parameters

Parameter	Qualifier	Cardinality	Content	Description
accFilter	M	1	Filter	This is used to specify one or more sub-system(s) within the accelerator for which the statistical data is to be reset.
accStatSelector	M	0..N	enum	It provides a list of attribute names of AccStatistics. If present, only statistics related to these attributes will be reset for the accelerator sub-system(s) matching the filter. If absent, all the related statistics of the chosen sub-system will be reset.

7.5.3.3 Output Parameters

None.

7.5.3.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the resetting of the statistical information in the accelerator has been reset according to the input parameters specified in the operation.

7.6 EPD Data Interface

7.6.1 Description

This interface allows the VNF to send data to an accelerator and receive data from an accelerator. Depending on the mode the accelerator is used in (i.e Look-aside, in-line, or offload), different operations are used.

NOTE: This clause only covers the case when VNFs and NFVI are in the same domain.

7.6.2 Accelerate operation

7.6.2.1 Description

This operation enables VNFs to send data to a look-aside accelerator and receive the result back.

Table 7.6.2.1-1 lists the information flow exchanged between the VNF and the NFVI.

Table 7.6.2.1-1: Accelerate Operation

Message	Requirement	Direction
AccDataRequest	Mandatory	VNF → NFVI
AccDataResponse	Mandatory	NFVI → VNF

7.6.2.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.6.2.2-1.

Table 7.6.2.2-1: Accelerate operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accData	M	1..N	AccDataType	Data for the Accelerator (including metadata).
accChannel	M	0…1	Integer	Specify the channel where data to be sent to.

7.6.2.3 Output Parameters

The output parameters received when invoking the operation shall follow the indications provided in table 7.6.2.3-1.

Table 7.6.2.3-1: Accelerate operation output parameters

Parameter	Qualifier	Cardinality	Content	Description
accData	M	1..N	AccDataType	Data received from the Accelerator

7.6.2.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the output contains the data received back from the accelerator.

7.6.3 AccSend operation

7.6.3.1 Description

This operation enables VNFs to send data to an in-line accelerator.

Table 7.6.3.1-1 lists the information flow exchanged between the VNF and the NFVI.

Table 7.6.3.1-1: AccSend Operation

Message	Requirement	Direction
AccSendDataRequest	Mandatory	VNF → NFVI
AccSendDataResponse	Mandatory	NFVI → VNF

7.6.3.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.6.3.2-1.

Table 7.6.3.2-1: AccSend operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accData	M	1..N	AccDataType	Data for the Accelerator.
accChannel	M	0…1	Integer	Specify the channel where data to be sent to.

7.6.3.3 Output Parameters

None.

7.6.3.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the data has been sent to the accelerator.

7.6.4 AccReceive operation

7.6.4.1 Description

This operation enables VNFs to receive data from an in-line accelerator.

Table 7.6.4.1-1 lists the information flow exchanged between the VNF and the NFVI.

Table 7.6.4.1-1: AccReceive Operation

Message	Requirement	Direction
AccReceiveDataRequest	Mandatory	VNF → NFVI
AccReceiveDataResponse	Mandatory	NFVI → VNF

7.6.4.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.6.4.2-1.

Table 7.6.4.2-1: AccReceive operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
maxNumberOfDataItems	M	1	Integer	Maximum number of data items to be received. See note.
accChannel	M	0…1	Integer	Specify the channel where data is requested from.

NOTE:
- As part of acceleration configuration process, terms are negotiated (payload, how it is framed, number of descriptors, etc.).
- VNF controls when the accelerated is allowed to send burst(s) of data (number of data items allowed are limited to the maxNumberOfDataItems parameter).
- VNFs could be interrupted when data items are available. VNFs could request accelerators to send the maxNumberOfDataItems whenever VNF is ready (and performs this operation) and accelerator has the data.
- A Data Item is defined as data that are related to each other (example data that are part of one image).
- Different Data Items may have same metadata but they are not related.

7.6.4.3 Output Parameters

The output parameters received when invoking the operation shall follow the indications provided in table 7.6.4.3-1.

Table 7.6.4.3-1: AccReceive operation output parameters

Parameter	Qualifier	Cardinality	Content	Description
accData	M	0..N	AccDataType	Data from the Accelerator (see note)
NOTE: Cardinality of 0 is used to indicate that no data was received.				

7.6.4.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; number of bursts received from the accelerator has been returned.

7.6.5 RegisterForAccDataAvailableEvent operation

7.6.5.1 Description

This operation enables the VNF to register for an event from the accelerator that indicate that data is available and ready to be retrieved. Table 7.6.5.1-1 lists the information flow exchanged between the VNF and the NFVI.

NOTE: RegisterForAccEvent operation within the management interface is capable of performing this operation in the same way. However, this operation is intended to be specific for Data Completion Event within the data interface itself for performance considerations.

Table 7.6.5.1-1: RegisterForAccDataAvailableEvent Operation

Message	Requirement	Direction
RegisterForAccDataAvailableEventRequest	Optional	VNF → NFVI
RegisterForAccDataAvailableEventResponse	Optional	NFVI → VNF

7.6.5.2 Input parameters

The input parameters when invoking the operation shall follow the indications provided in table 7.6.5.2-1.

Table 7.6.5.2-1: RegisterForAccDataAvailableEvent operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
regHandlerId	M	1	Identifier	Identifier of the registration.
accChannel	M	0…1	Integer	Specify the channel where event is requested for.

7.6.5.3 Output parameters

None.

7.6.5.4 Operation Result

After successful operation, the VNF is registered to receive notification related to availability of new data in the accelerator which has not previously been retrieved to by the VNF.

7.6.6 AccDataAvailableEventNotification operation

7.6.6.1 Description

This operation informs the VNF about data availability in the accelerator which the VNF shall registered for by using RegisterForAccDataAvailableEvent operation.

Table 7.6.6.1-1 lists the information flow exchanged between the VNF and the NFVI.

Table 7.6.6.1-1: AccDataAvailableEventNotification Operation

Message	Requirement	Direction
AccDataAvailableEventNotificationRequest	Optional	NFVI → VNF
AccDataAvailableEventNotificationResponse	Optional	VNF → NFVI

7.6.6.2 Input Parameters

The input parameters sent when invoking the operation shall follow the indications provided in table 7.6.6.2-1.

Table 7.6.6.2-1: AccDataAvailableEventNotification operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
regHandlerId	M	1	Identifier	This is reference used by VNF when registering for the data available event.

7.6.6.3 Output Parameters

None.

7.6.6.4 Operation Result

The result of the operation indicates if it has been successful or not with a standard success/error result.

In the case the operation is successful; the VNF has been successfully notified about the availability of the data in the accelerator.

7.6.7 DeRegisterForAccDataAvailableEvent operation

7.6.7.1 Description

This operation enables the VNF to de-register from a previously registered event for data availability in the acceleration.

Table 7.6.7.1-1 lists the information flow exchanged between the VNF and the NFVI

Table 7.6.7.1-1: DeRegisterForAccDataAvailableEvent Operation

Message	Requirement	Direction
DeRegisterForAccDataAvailableEventRequest	Optional	VNF → NFVI
DeRegisterForAccDataAvailableEventResponse	Optional	NFVI → VNF

7.6.7.2 Input Parameters

The input parameters when invoking the operation shall follow the indications provided in table 7.6.7.2-1.

Table 7.6.7.2-1: DeRegisterForAccDataAvailableEvent operation input parameters

Parameter	Qualifier	Cardinality	Content	Description
accChannel	M	0…1	Integer	Specify the channel where event is related.

7.6.7.3 Output parameters

None.

7.6.7.4 Operation Result

After successful operation, the VNF has been de-registered from receiving notification related to availability of new data in the accelerator.

8 Information elements exchanged

8.1 Introduction

This clause defines, or references, definitions of information elements used in the interfaces defined in the present document.

8.2 Information elements and notifications related to EPD Interface Management operations

8.2.1 Introduction

This clause defines information elements related to EPD interface management operation.

8.2.2 AccCapabilities information element

8.2.2.1 Description

This information element provides the details of the accelerator capabilities of an existing accelerator.

8.2.2.2 Attributes

The AccCapabilities information element shall follow the indications provided in table 8.2.2.2-1.

Table 8.2.2.2-1: Attributes of the AccCapabilities information element

Attribute	Qualifier	Cardianlity	Content	Description
subSysCapabilities	M	1..N	SubSysCapabilities	Details of the capabilities per sub-system
accCapabilities	M	0…N	KeyValuePair	Non-subsystem capabilities

8.2.3 SubSysCapabilities information element

8.2.3.1 Description

This information element provides the details of the accelerator sub-system capabilities of an existing accelerator.

8.2.3.2 Attributes

The SubSysCapabilities information element shall follow the indications provided in table 8.2.3.2-1.

Table 8.2.3.2-1: Attributes of the SubSysCapabilities information element

Attribute	Qualifier	Cardianlity	Content	Description
subSysName	M	1	string	Name of sub-system
userDefinedData	M	0… N	KeyValuePair	Key: Value pair

8.2.4 AccEventType information element

8.2.4.1 Description

This information element provides the details of the accelerator event type.

8.2.4.2 Attributes

The AccEventType information element shall follow the indications provided in table 8.2.4.2-1.

Table 8.2.4.2-1: Attributes of the AccEventType information element

Attribute	Qualifier	Cardianlity	Content	Description
accEventId	M	1	Identifier	Identify what event the VNF is interested in
accEvenFilter	M	0… N	KeyValuePair	Event filter

8.2.5 AccEventMetaData information element

8.2.5.1 Description

This information element provides the details of the accelerator event data notification returned by NFVI to VNF upon registering for an event.

8.2.5.2 Attributes

The AccEventMetaData information element shall follow the indications provided in table 8.2.5.2-1.

Table 8.2.5.2-1: Attributes of the AccEventMetaData information element

Attribute	Qualifier	Cardianlity	Content	Description
accDefinedData	M	0… N	KeyValuePair	Key: Value pair

8.3 Information elements and notifications related to EPD Interface Configuration operations

8.3.1 Introduction

This clause defines information elements related to EPD interface configuration operation.

8.3.2 AccConfigurationType information element

8.3.2.1 Description

This data type provides the list of attributes for the configuration of an accelerator and its sub-systems.

8.3.2.2 Attributes

The list of attributes is provided in table 8.3.2.2-1.

Table 8.3.2.2-1: AccConfigurationType attributes

Attribute	Qualifier	Cardianlity	Content	Description
accSubSysConfigurationData	M	1..N	AccSubSysConfigurationType	Sub-system configuration
configurationData	M	0… N	KeyValuePair	Key: Value pair (see note)
NOTE: ConfigurationData relates to configurations that don't belong to a particular sub-system.				

8.3.3 AccSubSysConfigurationType information element

8.3.3.1 Description

This data type provides the list of attributes for the configuration of a particular accelerator sub-system.

8.3.3.2 Attributes

The list of attributes is provided in table 8.3.3.2-1.

Table 8.3.3.2-1: AccSubSysConfigurationType attributes

Attribute	Qualifier	Cardianlity	Content	Description
subSysName	M	1	string	Name of sub-system
ConfigurationData	M	0… N	KeyValuePair	Key: Value pair

8.4 Information elements and notifications related to EPD Interface Monitoring operations

8.4.0 Introduction

This clause defines information elements related to EPD interface monitoring operation.

8.4.1 AccStatistics information element

8.4.1.1 Description

This information element provides the so far collected statistical information of an existing accelerator.

8.4.1.2 Attributes

The AccStatistics information element shall follow the indications provided in table 8.4.1.2-1.

Table 8.4.1.2-1: Attributes of the AccStatistics information element

Attribute	Qualifier	Cardianlity	Content	Description
subSysStatistics	M	1..N	SubSysStatistics	Collected statistics per sub-system
accStatisitcs	M	0…N	KeyValuePair	Non-subsystem statistics

8.4.2 SubSysStatistics information element

8.4.2.1 Description

This information element provides the so far collected statistical information of the accelerator sub-system of an existing accelerator.

8.4.2.2 Attributes

The SubSysStatistics information element shall follow the indications provided in table 8.4.2.2-1.

Table 8.4.2.2-1: Attributes of the SubSysStatistics information element

Attribute	Qualifier	Cardianlity	Content	Description
subSysName	M	1	string	Name of sub-system
userDefinedData	M	1… N	KeyValuePair	Key: Value pair, see note
NOTE: Example "Peak Throughput".				

8.5 Information elements and notifications related to EPD Data Interface operations

8.5.1 Introduction

This clause defines information elements related to EPD data interface operation.

8.5.2 AccDataType information element

8.5.2.1 Description

This data type provides the list of attributes for the send and receive data operations on an accelerator.

8.5.2.2 Attributes

The list of attributes is provided in table 8.5.2.2-1.

Table 8.5.2.2-1: AccDataType attributes

Attribute	Qualifier	Cardianlity	Content	Description
accMetadata	M	0 … N	KeyValuePair	Accelerator defined metadata
size	M	1	Integer	The size of data in bytes
data	M	1	Binary	The actual data

Annex A (informative): Authors & contributors

The following people have contributed to the present document:

Rapporteur:

- Abdel Hafiz RABI, Intel

Other contributors:

- Bruno Chatras, Orange
- Rob Diamond, ARM
- Bob Monkman, ARM
- Brian Skerry, Intel
- Jinwei Xia, Huawei
- Andrew Thurber, Cisco
- Rabi Abdel, Altera
- AiJuan Feng, Huawei
- JianXing Hou, Huawei
- Qian Wang, China Telecom
- Yunpeng Xie, China Telecom
- Dan Daly, Intel
- Jan Ignatius, Nokia Networks
- Michaek Rooke, Nokia
- Allen Chen, Intel
- Rongwei Ren , China Mobile

Annex B (informative): Bibliography

- ETSI GS NFV-INF 004: "Network Functions Virtualisation (NFV); Infrastructure; Hypervisor Domain".
- Microsoft®: Network Driver Interface Specification (NDIS) version 6.0; IPSec Offload Version 2.

NOTE: Available at https://msdn.microsoft.com/library/windows/hardware/ff556996.aspx.

- ETSI GS NFV-INF 001: "Network Functions Virtualisation (NFV); Infrastructure Overview".
- NVM Express™ Inc: "NVM Express 1.0e, NVM Express 1.1, NVM Express 1.2".

NOTE: Available at http://www.nvmexpress.org/specifications/.

Annex C (informative): Change History

Date	Version	Information about changes
2016-April	2.1.2	Implementation of Extended TOC
2016-May	2.1.3	Changes From NFV#14
2016-AUG	2.1.4	Contributions from IFA S2#35 meeting
2016-AUG	2.1.5	Contributions from IFA S2#38 meeting
2016-SEP	2.1.6	Contributions from IFA S2#39 meeting
2016-OCT	2.1.7	Contributions from NFV#15
2016-NOV	2.1.8	Contributions from IFAS2#42, NFVIFA#40 F2F
2016-NOV	2.1.9	Contributions from IFAS2#43
2016-DEC	2.1.10	Contributions from IFA S2#44, IFA S2#45
2017-JAN	2.1.11	Contributions from NFV#16
2017-JAN	2.1.12	Contributions from IFAS2#47

History

Document history		
V2.1.1	March 2016	Publication
V2.3.1	August 2017	Publication